

Efficient Parallel Algorithms with Linear Queries for Non-Monotone Submodular Maximization

Canh V. Pham^{1*}, Tan D. Tran¹, Dung T. K. Ha¹ and My T. Thai²

¹ORLab, Faculty of Computer Science

Phenikaa School of Computing, Phenikaa University, Hanoi, Vietnam

²Department of Computer and Information Science and Engineering
University of Florida, Gainesville, Florida 32611

{canh.phamvan, tan.trandin, dung.hathikim}@phenikaa-uni.edu.vn, mythai@cise.ufl.edu

Abstract

In this work, we propose the first constant-approximation algorithms, LinAst and LinAtg, which simultaneously achieve optimal query complexity $O(n)$ and adaptive complexity $O(\log n)$ for non-monotone submodular maximization under a cardinality constraint k over a ground set of size n . Specifically, compared with existing algorithms that attain the best known adaptive complexity of $O(\log n)$, our approach preserves this adaptivity while reducing the query complexity from $O(n \log k)$ to $O(n)$ and improving the approximation ratio from $0.172 - \epsilon$ to $0.193 - \epsilon$. Our algorithms are built upon LinAdapt, which achieves a constant approximation ratio with $O(\log n)$ adaptive rounds and linear query complexity by requiring only $O(1)$ candidate guesses of the optimal value. We further introduce the BoostAdapt algorithm, which improves the approximation guarantee to $0.25 - \epsilon$ with $O(\log n \log k)$ adaptive complexity and $O(n \log k)$ query complexity, based on a novel staggered greedy threshold framework that alternately constructs two disjoint solution sets over $O(\log k)$ sequential rounds. Extensive experiments on standard benchmark datasets demonstrate that our algorithms consistently outperform state-of-the-art methods in terms of solution quality, query complexity, and running time.

1 Introduction

Maximizing a non-negative (not necessarily monotone) submodular set function under a cardinality constraint is a fundamental problem with broad applications in artificial intelligence and machine learning. Notable examples include data summarization [Kuhnle, 2021a; Lin and Bilmes, 2011; Fahrback *et al.*, 2019a; Han *et al.*, 2021; Mirzasoleiman *et al.*, 2016], revenue maximization in social networks [Kuhnle, 2021a; Chen and Kuhnle, 2023], recommendation systems [Mirzasoleiman *et al.*, 2016], and weight cut problems [Chen and Kuhnle, 2023; Kuhnle, 2021a].

Formally, given a finite ground set V of size n , a submodular set function $f : 2^V \rightarrow \mathbb{R}^+$, and a positive integer k representing the cardinality constraint, the goal of the Submodular Maximization under a Cardinality Constraint (SMC) problem is to find a subset $S \subseteq V$ such that $|S| \leq k$ and $f(S)$ is maximized. This problem has attracted extensive attention, with many works focusing on developing approximation algorithms with provable performance guarantees [Buchbinder *et al.*, 2014; Kuhnle, 2019; Chen and Kuhnle, 2023].

However, the exponential growth of the solution space with the input size poses a significant computational challenge, particularly in modern large-scale applications. This has spurred growing interest in designing efficient parallel algorithms capable of leveraging modern parallel computing architectures to compute high-quality solutions rapidly (see Table 2 for a summary of parallelizable methods). In this context, the concept of adaptive complexity (or adaptivity) has emerged as a key metric for assessing the parallel efficiency of such algorithms. A central notion in assessing the parallel efficiency of such algorithms is adaptive complexity (or adaptivity), which measures the number of sequential rounds an algorithm requires, allowing a polynomial number of oracle queries to be processed in parallel during each round [Balkanski and Singer, 2018]. Notably, reducing the adaptive complexity from $O(\log^2 n)$ to $O(\log n)$ has been shown to significantly cut down the number of sequential steps, resulting in notable improvements in practical runtime performance [Fahrback *et al.*, 2019a; Ene and Nguyen, 2020; Amanatidis *et al.*, 2021; Fahrback *et al.*, 2023; Kuhnle, 2021a; Chen and Kuhnle, 2024]. Although recent advances have successfully brought down the adaptive complexity to $O(\log n)$, several significant challenges remain: (1) The high query complexity made the some parallel algorithms with $O(\log n)$ adaptive complexity impractical for real applications. For instances, [Chen and Kuhnle, 2024] demonstrated that the algorithm in [Ene and Nguyen, 2020] providing the best ratio of $1/e - \epsilon$ within $O(\log n)$ adaptivity is of mostly theoretical interest because it needs $\Omega(nk^2 \log^2(n))$ queries to approximate the multi-linear extension of a submodular function and its gradient.

(2) The approximation ratios of the existing practical parallel algorithms [Fahrback *et al.*, 2019a; Fahrback *et al.*, 2023; Kuhnle, 2021a; Chen and Kuhnle, 2024; Cui *et al.*, 2023] are

*Corresponding author

Table 1: A comparison of efficient parallel algorithms for SMC is presented across three criteria: approximation ratio, adaptive complexity, and query complexity. The best known results are highlighted in **bold**.

Reference	Appr. Ratio	Adaptivity	Query Complexity
[Buchbinder <i>et al.</i> , 2015]	$1/e - \epsilon$	$O(k)$	$O(n)$
[Buchbinder and Feldman, 2024]	$0.385 - \epsilon$	$O(n)$	$O(n + k^2)$
[Chen <i>et al.</i> , 2024b]	$0.385 - \epsilon$	$O(n)$	$O(kn)$
[Buchbinder and Feldman, 2024]	0.401	$\text{poly}(n)$	$\text{poly}(n)$
Blits [Balkanski <i>et al.</i> , 2018]	$1/(2e) - \epsilon$	$O(\log^2(n))$	$O(\text{opt}n \log^2(n) \log(k))$
[Chekuri and Quanrud, 2019]	$3 - 2\sqrt{2} - \epsilon$	$O(\log^2(n))$	$O(nk^4 \log^2(n))$
[Ene and Nguyen, 2020]	$1/e - \epsilon$	$O(\log n)$	$\Omega(nk^2 \log^2(n))$
ANM [Fahrbach <i>et al.</i> , 2023]	$0.039 - \epsilon$	$O(\log n)$	$O(n \log(k))$
ParKnapSack[Amanatidis <i>et al.</i> , 2021]	$0.172 - \epsilon$	$O(\log n)$	$O(nk \log(n) \log(k))$
AST [Chen and Kuhnle, 2024]	$1/6 - \epsilon$	$O(\log n)$	$O(n \log(k))$
ATG [Chen and Kuhnle, 2024]	$0.193 - \epsilon$	$O(\log(n) \log(k))$	$O(n \log(k))$
ParSKP [Cui <i>et al.</i> , 2023]	$1/8 - \epsilon$	$O(\log(n) \log(k)) $ $O(\log n)$	$O(n \log^2(n) \log(k)) O(nk \log^2(n))$
ParSSP [Cui <i>et al.</i> , 2023]	$1/4 - \epsilon$	$O(\log^2(n) \log(k)) O(\log^2(n))$	$O(n \log^2(n) \log(k)) O(nk \log^2(n))$
PIG [Chen <i>et al.</i> , 2025]	0.25	$O(\log(n) \log(k))$	$O(n \log(n) \log(k))$
PITG [Chen <i>et al.</i> , 2025]	$1/e - \epsilon$	$O(\log(n) \log(k))$	$O(n \log(n) \log(k))$
LinAst (this work)	$1/6 - \epsilon$	$O(\log n)$	$O(n)$
LinAtg (this work)	$0.193 - \epsilon$	$O(\log n)$	$O(n)$
BoostAdapt (this work)	$0.25 - \epsilon$	$O(\log(n) \log(k))$	$O(n \log(k))$

still low as they have a large gap in the approximation ratios compared to the best non-parallel algorithm for SMC (e.g. the ratio of 0.401 in [Buchbinder and Feldman, 2024]). This raises two interesting questions for us to solve SMC: *Can we improve the query complexity and approximation ratio of a practical parallel algorithm?*

Our contributions and techniques. In this work, we tackle the above questions with the following contributions:

First, we introduce LinAdapt, the first constant approximation ratio ($1/(12 + O(\epsilon))$) within near-optimal adaptivity $O(\log n)$ and linear query complexity $O(n)$, where $\epsilon > 0$ is a constant parameter. LinAdapt is used as a building block to reduce the query complexity of our latter algorithms. The key idea to obtain a such improvement lies in: **(1)** constructing two sets S, S' in $O(\log n)$ sequential rounds with an interesting property: $f(X \cup S) \leq O(1) \cdot f(S')$ with high probability for any set $X \subseteq V, |X| \leq k$, and S' with $|S'| \leq k$ is the set of last elements added into S ; **(2)** combining **(1)** appropriately with the unconstrained submodular maximization algorithm to get the desired ratio.

Second, we introduce two algorithms LinAst and LinAtg which run $O(\log n)$ adaptivity and $O(n)$ query complexity and return the approximation ratios to $1/6 - \epsilon$ and $0.193 - \epsilon$, respectively. Therefore, our algorithm LinAtg improves the approximation ratio from $0.172 - \epsilon$ to $0.193 - \epsilon$ and significantly reduces the query complexity of the current best practical algorithm of [Amanatidis *et al.*, 2021].

Third, we propose a practical algorithm, denoted by BoostAdapt, that achieves an approximation ratio of $1/4 - \epsilon$ with $O(\log n \log k)$ adaptive complexity and $O(n \log k)$ query complexity. As a result, BoostAdapt improves the best-known approximation ratio for algorithms with the **same adaptive and query complexity** from $0.193 - \epsilon$ to $0.25 - \epsilon$. The design of BoostAdapt is inspired by the twin greedy strategy, which incorporates ThreshSeq to update two disjoint sets, X and Y , in an alternating manner over only $O(\log k)$ iterations. It must be noted that the staggered

threshold is different from the Iterated Greedy [Gupta *et al.*, 2010], which uses two threshold greedy strategies to construct candidate solutions separately.

Finally, to show the consistency between theory and practice, we conducted extensive experiments on two applications: Revenue Maximization and Maximum Weighted Cut. The results show that our algorithms not only produce the better solution quality and query complexity sharply but also provide comparative adaptivity to state-of-the-art algorithms. **Paper Organization.** The rest of this work is structured as follows. Section 2 provides the literature review on the studied problem. Notations are presented in Section 3. Sections 4-6 introduce our algorithms and theoretical analysis. Experimental computation is provided in Section 7. Finally, Section 8 concludes this work.

2 Related Works

Approximation Algorithms. The first popular approach to solving the SMC problem in practice is to design approximation algorithms with low query complexity. The greedy algorithm was an effective approach for submodular optimization problems. It sequentially selects elements with the largest marginal gain and explores the diminishing return property to get performance bounds. [Gupta *et al.*, 2010] first developed an iterated greedy algorithm with $1/6$ ratio in $O(nk)$ queries for SMC. The work of [Lee *et al.*, 2010] latter improve the ratio to $1/4 - \epsilon$ by developing a local search method but this wasted an expensive query complexity of $O(n^4 \log n)$. Significantly, the elegant random greedy in [Buchbinder *et al.*, 2014] could provide the ratio of $1/e$ with $O(nk)$ query complexity. Instead of selecting an element with the best marginal gain as the naive greedy, it chose a uniformly random element from the set of k elements with the largest marginal gain. [Buchbinder *et al.*, 2015] then improved the query complexity to $O(n \log(1/\epsilon)/\epsilon^2)$ but still kept the same $1/e$ ratio. Recently, independent works [Chen *et al.*, 2025] and [Tukan *et al.*, 2024] improved the ratio to $0.385 - \epsilon$ in $O(n^2)$ queries

and $O(n)$ adaptive complexity. To the best of our knowledge, the best ratio for SMC was 0.401 by [Buchbinder and Feldman, 2024]. However, this work used the multi-linear extensions method and used a considerably high query complexity of $O(n^5)$ [Chen and Kuhnle, 2023]. Finally, it must be emphasized that the above works weren't well parallelized due to their high adaptive complexities of $\Omega(n)$.

Parallel Algorithms. Research on parallel algorithms was initiated by [Balkanski and Singer, 2018]. In this seminal work, they introduced the concept of *adaptive complexity or adaptivity* that measures the parallelizability of algorithm, showed a lower bound on adaptive complexity of $O(\log(n)/\log(\log n))$ to achieve a constant ratio and devised an $1/3$ -approximation algorithm in $O(\log n)$ adaptive rounds for the monotone SMC problem. Since then, many works focused on devising algorithms with low adaptivity with tighter ratio for SMC [Balkanski et al., 2019; Fahrback et al., 2019b; Ene et al., 2019; Chen et al., 2021]. The best parallel algorithm for monotone SMC had an optimal ratio of $1 - 1/e - \epsilon$ in $O(\log n)$ adaptivity and $O(n)$ queries was due to [Chen et al., 2021]. However, their performance bounds relied on the monotone objective and did not hold for non-monotone.

For the non-monotone SMC, [Chekuri and Quanrud, 2019; Balkanski et al., 2018] first developed parallelizable algorithms with $3 - 2\sqrt{2} - \epsilon$ and $1/(2e) - \epsilon$ ratios, respectively; both took $O(\log^2 n)$ adaptivity. [Ene and Nguyen, 2020] improved the ratio to $1/e - \epsilon$ in nearly optimal adaptive complexity of $O(\log n)$. However, due to the high query complexity of $\Omega(nk^2 \log^2 n)$ to approximate the multilinear extension of a submodular function and its gradient, the [Ene and Nguyen, 2020]' algorithm and algorithms based on multilinear extension methods in general are still impractical in some real applications [Kuhnle, 2021a; Fahrback et al., 2019a; Chen and Kuhnle, 2024; Cui et al., 2023]. To attack the above issue, [Fahrback et al., 2019a] first aimed to reduce the query complexity to near-linear of $O(n \log k)$ and still kept the $O(\log n)$ adaptive complexity. However, their algorithm resulted in a small ratio, $0.039 - \epsilon$. In subsequent work, [Kuhnle, 2021a] tried to boost the ratio to $1/6 - \epsilon$ in $O(\log n)$ adaptive rounds with $O(n \log k)$ queries. Their algorithm further improved the ratio to $0.193 - \epsilon$ in $O(\log^2 n)$ adaptive rounds by exploring iterated greedy framework in [Gupta et al., 2010]. It must be noted that the ratios of both [Fahrback et al., 2019a] and [Kuhnle, 2021a] may not hold because they adapt the threshold sampling routine in [Fahrback et al., 2019b], which was pointed out to be disabled with non-monotone functions [Chen and Kuhnle, 2024]. Recently, [Chen and Kuhnle, 2024; Fahrback et al., 2023] recovered the ratios of [Fahrback et al., 2019a; Kuhnle, 2021a] by some threshold sampling routines with the respective analysis. [Cui et al., 2023] then showed two algorithms with ratios of $1/8 - \epsilon$ and $1/4 - \epsilon$ with $O(\log n)$ and $O(\log^2 n)$ adaptive complexities, respectively. However, they took at least $\Omega(nk \log^2 n)$ query complexity. More recently, [Chen et al., 2025] proposed two parallel algorithms, PIG and PITG, which achieve approximation ratios of 0.25 and $1/e - \epsilon$, respectively. Both algorithms take a query complexity of $O(n \log(n) \log(k))$ and an adaptive complexity of $O(\log(n) \log(k))$.

3 Preliminaries

Given a ground set V of size n , and the set function $f : 2^V \mapsto \mathbb{R}^+$ is submodular iff it satisfies the diminishing return property, i.e., $f(A \cup \{e\}) - f(A) \geq f(B \cup \{e\}) - f(B)$ where $A \subseteq B$ and $e \notin B$. The marginal gain (or contribution gain) of a set T to a set S is defined as $f(T|S) = f(T \cup S) - f(S)$. For simplicity, we denote $f(\{e\}|A)$ by $f(e|A)$ and assume f normalized, i.e., $f(\emptyset) = 0$. Given a positive number k (cardinality constraint), the SMC problem is to determine $\arg \max_{S \subseteq V, |S| \leq k} f(S)$.

We define $[k] = \{1, 2, \dots, k\}$ for any integer k . We denote an instance of SMC by a tuple (f, V, k) and O is an optimal solution with the optimal value $\text{opt} = f(O)$. In this work, we assume that there exists an oracle query that returns $f(S)$ when queried for the set S with any $S \subseteq V$. The parallelization capacity of an algorithm is evaluated through the following definition.

Definition 1 ([Balkanski and Singer, 2018]). *The adaptivity or adaptive complexity of an algorithm is the minimum sequential number of rounds needed such that in each round the algorithm makes $O(\text{poly}(n))$ independent queries to the evaluation oracle.*

We recap two sub-problems for solving SMC in parallel setting, Batch Selection with Threshold (BST) and unconstrained submodular maximization (USM). Given an instance (f, V, B) , a fixed threshold τ and $\epsilon > 0$ as inputs, BST asks to find a subset $S \subseteq V$ in $O(\log n)$ adaptive complexity satisfying two conditions: (1) $f(S) \geq (1 - \epsilon)\tau|S|$; (2) if $|S| < k$, $f(e|S) < \tau$ for any $e \notin S$.

Although several attempts exist to solve the BST, most only work with a monotone submodular function [Fahrback et al., 2019b; Kazemi et al., 2019]. In this work, we use the ThreshSeq algorithm in [Chen and Kuhnle, 2024] and its theoretical satisfaction to analyze our algorithm's performance in a nonmonotone setting.

Theorem 1 (Theorem 3 in [Chen and Kuhnle, 2024]). *Let (f, V, k) be an instance of SMC. For any constants $\epsilon, \delta > 0$, the algorithm ThreshSeq outputs $A' \subseteq A \subseteq V$ such that the following properties hold: 1) The algorithm succeeds with probability at least $1 - \delta/n$. 2) There are $O(n/\epsilon)$ oracle queries in expectation and $O(\log(n/\delta)/\epsilon)$ adaptive complexity. 3) It holds $f(A') \geq (1 - \epsilon)\tau|A|$. If $|A| < k$, then $f(e|A) < \tau$ for all $e \in V$. 4) It also holds $f(A') \geq f(A)$ and $|A'| \geq (1 - \epsilon)|A|$.*

Regarding USM, which aims to find $\arg \max_{S \subseteq V} f(S)$, there exist several solving methods giving the constant approximation ratios such as the [Fahrback et al., 2019a]'s algorithm (USM1) provides a ratio of $1/4 - \epsilon$ with probability at least $1 - \delta$ for the problem in $O(1)$ adaptive round and $O(\frac{1}{\epsilon} \log(\frac{1}{\delta}))$ queries and an essentially optimal algorithm of [Chen et al., 2019] (USM2) slightly improves the approximation ratio to $1/2 - \epsilon$ using $O(\log(1/\epsilon)/\epsilon)$ adaptive rounds and $O(n \log^3(1/\epsilon)/\epsilon^4)$ query complexity. Due to the space limitations, the pseudocodes for missed algorithm are given in the appendix.

4 A Parallel Algorithm with Linear Queries

In this section, we introduce LinAdapt (Algorithm 1), the first constant ratio approximation algorithm within $O(\log n)$ adaptivity and $O(n)$ query complexity. LinAdapt receives

Algorithm 1: LinAdapt($f, V, k, \alpha, \epsilon, \delta$)

- 1: **Input:** $f, V, k, \alpha, \epsilon, \delta$.
 - 2: $A, A' \leftarrow \text{LinBoundSet}(f, V, k, \alpha, \epsilon, \alpha, \delta/3)$
 - 3: $B, B' \leftarrow \text{LinBoundSet}(f, V \setminus A, k, \alpha, \epsilon, \delta/3)$
 - 4: $C \leftarrow \text{USM1}(A, \epsilon, \delta/(3n))$
 - 5: **Return** $\arg \max_{S \in \{A', B', C\}} f(S)$
-

an instance (f, V, k) , accuracy parameters $\epsilon, \delta > 0$, a constant $\alpha > 0$ and works in a novel algorithmic framework. LinAdapt first sequentially calls a subroutine LinBoundSet twice to find two candidates: one to get A over the ground set V and the other to get B' over a new ground set $V \setminus A$. Note that A and B in LinAdapt are temporary sets that are useful only for theoretical analysis. For any ground set V , LinBoundSet is a key subroutine with the following nice properties:

- 1) running in $O(\log(n)/\epsilon^2)$ and $O(n/\epsilon^3)$ query complexity.
- 2) returning two sets A and A' so that $f(A \cup X) \leq (8 + 2\alpha + \frac{2}{\alpha} + O(\epsilon))f(A')$, for any subset $X \subseteq V, |X| \leq k$ with high probability.

To easily follow the algorithm, we will latter provide a detailed analysis of LinBoundSet in Section 4. Finally, LinAdapt calls USM1 to find another candidate solution C and returns the best among A', B', C . Conceptually, our LinAdapt takes advantage of the properties LinBoundSet (by Theorem 3), and the fact that $f(O) \leq f(O \cup A) + f(O \cup B)$ to give the desired bound of solution. We state the LinAdapt's performance in Theorem 2.

Theorem 2. For any input (f, V, k) , where $\epsilon, \delta \in (0, 1)$, LinAdapt runs in $O(\log(n/\delta)/\epsilon^2)$ adaptive complexity and $O(n/\epsilon^3)$ query complexity, returns a solution S satisfying $\text{opt} \leq (8 + 2\alpha + \frac{2}{\alpha} + (\frac{16}{1-4\epsilon} + 2(1 + \frac{1}{\alpha}) \frac{2(2-\epsilon)}{(1-\epsilon)(1-2\epsilon)}))\epsilon f(S)$ with probability at least $1 - \delta/n$. If $\alpha = 1$, the algorithm achieves the best ratio of $\frac{1}{12+O(\epsilon)}$.

Proof. Each subroutine terminates with failure probability $1/(3n)$. By the union bound of probabilities, the failure probability of the algorithm is bounded by $3 \cdot \frac{\delta}{3n} = \frac{\delta}{n}$. Let $a = 2 + \alpha + \frac{1}{\alpha} + (1 + \frac{1}{\alpha}) \frac{2(2-\epsilon)}{(1-\epsilon)(1-2\epsilon)}\epsilon$. If the algorithm terminates successfully, we have:

$$f(O) \leq f(O \cup A) + f(O \cup B) \quad (1)$$

$$\leq f(O \cup A) + f((O \cup B) \setminus A) + f((O \cup B) \cap A) \quad (2)$$

$$= f(O \cup A) + f((O \setminus A) \cup B) + f(O \cap A) \quad (3)$$

$$\leq a(f(A') + f(B')) + \frac{4}{1-4\epsilon}f(C) \quad (4)$$

$$\leq (2a + \frac{4}{1-4\epsilon})f(S)$$

where inequality (1) is due to the submodularity of f and the fact that $A \cap B = \emptyset$; the inequality (2) is due to the submodularity of f , and the inequality (4) is due to Theorem 3 and USM1's performance. For the complexities, the algorithm calls USM1 and LinAdapt twice so its adaptive complexity is $2 \cdot O(\frac{\log n}{\epsilon^2}) + 1 = O(\frac{\log n}{\epsilon^2})$ and query complexity is $2 \cdot O(\frac{n}{\epsilon^3}) + O(\frac{1}{\epsilon} \log(\frac{n}{\epsilon})) = O(\frac{n}{\epsilon^3})$. $\square$

Algorithm 2: LinBoundSet($f, V, k, \alpha, \epsilon, \delta$)

- 1: **Input:** $f, V, k, \alpha, \epsilon, \delta$
 - 2: $e_{max} \leftarrow \max_{e \in V} f(e)$, $S \leftarrow \{e_{max}\}$,
 $\beta \leftarrow \epsilon \log((1 - e^{-\frac{\epsilon}{2}})/8)/16$, $\ell \leftarrow \lceil (4 + \frac{4}{\beta\epsilon}) \log(\frac{n}{\delta}) \rceil$
 - 3: **for** $j \leftarrow 1$ **to** ℓ **do**
 - 4: $V \leftarrow \{x \in V : f(x|S) \geq \alpha f(S)/k\}$
 - 5: **If** $V = \emptyset$ **then break;**
 - 6: $V = \{v_1, v_2, \dots, v_{|V|}\} \leftarrow \text{rand-permutation}(V)$
 - 7: $\Lambda_1 \leftarrow \{ \lfloor (1+\epsilon)^l \rfloor : 1 \leq \lfloor (1+\epsilon)^l \rfloor \leq k, l \in \mathbb{N} \}$
 - 8: $\Lambda_2 \leftarrow \{ \lfloor k + l\epsilon k \rfloor : \lfloor k + l\epsilon k \rfloor \leq |V|, l \in \mathbb{N} \} \cup \{|V|\}$
 - 9: $\Lambda = \{\lambda_1, \dots, \lambda_m\} \leftarrow \Lambda_1 \cup \Lambda_2$,
 $T_i \leftarrow \{v_1, v_2, \dots, v_i\}, T'_{\lambda_i} \leftarrow T_{\lambda_i} \setminus T_{\lambda_{i-1}}$
 - 10: Calculate $M_{\lambda_i} \leftarrow \max_{\lambda_j \leq \lambda_i} f(S \cup T_{\lambda_j})$ in parallel
 - 11: $b[v_i] \leftarrow \text{none}, \forall v_i \in V; B[\lambda_i] \leftarrow \text{false}, \forall \lambda_i \in \Lambda$
 - 12: **for** $\lambda_i \in \Lambda$ (in parallel) **do**
 - 13: **for** $v_l \in T'_{\lambda_i}$ (in parallel) **do**
 - 14: **if** $f(v_l|S \cup T_{\lambda_{i-1}}) \geq (1-\epsilon)\alpha M_{\lambda_{i-1}}/k$ **then**
 $b[v_l] \leftarrow \text{true}$
 - 15: **else if** $f(v_l|S \cup T_{\lambda_{i-1}}) < 0$ **then** $b[v_l] \leftarrow \text{false}$
 - 16: **end for**
 - 17: **if** $|\{v \in T'_{\lambda_i} : b[v] = \text{true}\}| \geq (1-\epsilon)|T'_{\lambda_i}|$ **then**
 $B[\lambda_i] \leftarrow \text{true}$
 - 18: **end for**
 - 19: $\lambda_1^* \leftarrow \max\{\lambda_i \in \Lambda, \lambda_i < k : B[\lambda_i] = \text{false}, B[\lambda_1] = B[\lambda_2] = \dots = B[\lambda_{i-1}] = \text{true}\}$
 - 20: $\lambda_2^* \leftarrow \max\{\lambda_i \in \Lambda, \lambda_i \geq k : \exists u \geq$
 $1 \text{ s.t. } |\bigcup_{j=u}^{i-1} T'_{\lambda_j}| \geq k \text{ and } (B[\lambda_u] = B[\lambda_2] = \dots = B[\lambda_{i-1}] = \text{true})\}$
 - 21: $\lambda^* \leftarrow \max\{\lambda_1^*, \lambda_2^*\}$
 - 22: Define new blocks $T''_{\lambda_i} \leftarrow \{v \in T'_{\lambda_i} : b[v] \neq \text{false}\}$
 - 23: $T^j \leftarrow \bigcup_{\lambda_i \leq \lambda^*} T''_{\lambda_i}, S \leftarrow S \cup T^j$,
 $V \leftarrow V \setminus \bigcup_{\lambda_i \leq \lambda^*} T'_{\lambda_i}$
 - 24: **end for**
 - 25: **If** $|V| > 0$, **then Return failure**
 - 26: $S' \leftarrow$ last blocks added into S with the size at most k
 - 27: **Return** S, S'
-

A key subroutine LinBoundSet (Algorithm 2). It receives an instance (f, V, k) , accuracy parameters $\epsilon, \delta > 0$ and a positive constant α . It first initiates a set S that contains an element with maximal value e_{max} and then operates in $\ell = O(\log n)$ iterations of the main loop (Lines 3-24). Denote S_j as S after iteration j . At each iteration of main loop, it generates a random permutation of V (Line 6) and divides V into blocks $T'_{\lambda_i}, \forall \lambda_i \in \Lambda$. We define an element $v_t \in T'_{\lambda_i}$ that is **good** if $f(v_t|\{v_1, \dots, v_{t-1}\}) \geq \alpha M_{\lambda_i}/k$, where $M_{\lambda_i} = \max_{j=0 \dots i} f(S \cup T_{\lambda_j}), i \in [m]$; a block T'_{λ_i} is **good** if it has at least $(1-\epsilon)\alpha|T'_{\lambda_i}|$ good elements.

At a high level, at iteration j , the algorithm selects a segment of elements T_{λ^*} and its subset T^j (Line 23) from set T , a random permutation of V satisfying three following requirements with high probability: (1) $|T^j| \geq (1 - \epsilon)|T_{\lambda^*}|$; (2) $f(T^j|S) \geq (1 - O(\epsilon))\alpha|T_{\lambda^*}|f(S)/k$; and (3) remove elements in V with the marginal gain is less than $\alpha f(S)/k$, i.e., $f(e|S) < \alpha f(S)/k$ for all $e \in V \setminus S$.

To achieve the requirements, we propose the strategy to select a pair of good subset S and S' in algorithm 2 as follows: It first removes every element whose marginal gain is less than $\alpha f(S)/k$ (Line 4); later, it selects T_{λ^*} , a consecutive sequence of elements T that contains at least $(1 - \epsilon)$ -fraction number of blocks that are good by finding λ^* in Line 19. Then, it selects new block T_{λ_i}'' that only contains elements with non-negative marginal gain from T_{λ_i}' and selects all new blocks from T_{λ^*} . At the end of each iteration, it adds T_j into S and the main loop ends after ℓ iterations or V is empty. Finally, the algorithm returns S' as the union of last T_{λ_i}'' blocks added to S so that the size of S' does not exceed k (Line 23).

By combining the our selection strategy for subsets S and S' with the analysis of probability bounds in [Chen *et al.*, 2021; Chen and Kuhnle, 2024], we provide the analysis LinBoundSet's performance in following process. We first focus on the bridge between S and S' in Lemma 1.

Lemma 1. *If LinBoundSet ends successfully, then $f(S) \leq \left(1 + \frac{1+\epsilon}{(1-\epsilon)(1-2\epsilon)}\right)f(S')$.*

We define an *iteration j succeeds* if the algorithm successfully filters out more than $\beta\epsilon$ -fraction of V at the next iteration. Otherwise, the iteration j *fails*. The following Lemma provides a bound of probability for the event that iteration j fails.

Lemma 2. $\Pr[\text{iteration } j \text{ fails}] \leq 1/2$.

Combine Lemma 1 and Lemma 2, we state the performance of LinBoundSet in Theorem 3.

Theorem 3. *For any $\epsilon, \delta \in (0, 1)$, the algorithm LinBoundSet runs in $O(\log(n/\delta)/\epsilon^2)$ queries, returns two sets S and S' such that the following holds unconditionally: (1) The algorithm succeeds with a probability of at least $1 - \delta/n$; (2) The algorithm takes $O(n/\epsilon^3)$ query complexity in expectation; (3) If the algorithm succeeds, it returns S, S' satisfying: for any subset $X \subseteq V, |X| \leq k$, we have $f(S \cup X) \leq \left(2 + \alpha + \frac{1}{\alpha} + (1 + \frac{1}{\alpha})\frac{2(2-\epsilon)}{(1-\epsilon)(1-2\epsilon)}\epsilon\right)f(S')$.*

5 Improved Ratio within Linear Queries

This section introduces two combinatorial algorithms within linear query and near-optimal adaptivity complexities: LinAtg and LinAst. Both share a common framework in which LinAdapt plays a central role and provides $O(1)$ number of guesses of the optimal value. Firstly, they adapt LinAdapt with $\alpha = 1, \delta = 1/3$ to give a candidate solution S_0 with $\text{opt} \in [f(S_0), (12 + O(\epsilon))f(S_0)]$. It thus provides $O(\frac{1}{\epsilon} \log(\frac{1}{\epsilon}))$ guesses of opt . They then use the Adaptive Simple Threshold (AST) framework [Chen and Kuhnle, 2024] and the Iterated Greedy (IG) algorithm framework [Gupta *et al.*, 2010; Chen and Kuhnle, 2024] to boost the ratio.

LinAst constructs several solutions in the main loop with $O(\frac{1}{\epsilon} \log(\frac{1}{\epsilon}))$ adaptive rounds: each iteration i calls ThreshSeq twice sequentially to get candidate solutions A_i', B_i' . It finally combines the obtained solution and the solution of USM over $\mathcal{A}_i'$ to get the final solution. LinAtg works in a different way. It sequentially constructs two pairs of disjoint sets (A, B) and (A', B') in two main loops which contain at most $\ell = O(\frac{1}{\epsilon} \log(\frac{1}{\epsilon}))$ iterations: each iteration calls ThreshSeq to select two batches of elements with high marginal gain (S, S') and adds them into (A, A') (or (B, B')), respectively. It also the solution of USM over $\mathcal{A}'$ and finally returns the best one among the obtained feasible solutions. We provide the performance of LinAtg and LinAst in Theorems 4, 5.

Theorem 4. *For any input (f, V, k, ϵ) , where $\epsilon \in (0, 1)$, LinAst runs in $O(\log(n)/\epsilon^2)$ adaptive complexity and $O(n/\epsilon^3)$ query complexity and returns an approximation ratio of $1/6 - \epsilon$ in expectation with probability of at least $1 - 1/n$.*

Theorem 5. *For any input (f, V, k, ϵ) , where $\epsilon \in (0, 1)$, LinAtg runs in $O(\log(1/\epsilon) \log(n)/\epsilon^2)$ adaptive complexity and $O(n/\epsilon^3)$ query complexity, returns an approximation ratio of $0.193 - \epsilon$ in expectation with probability of at least $1 - 1/n$.*

6 Boost Ratio with Near-Linear Queries

This section introduces our last algorithm, BoostAdapt (Algorithm 3), which further improves the ratio to $1/4 - \epsilon$ in $O(\log(n) \log(k))$ adaptivity and $O(n \log(k))$ query complexity. Different from LinAtg and LinAst, after reusing LinAdapt's solution (Line 2), BoostAdapt operates in a novel staggered strategy that consists of a main loop with $O(\log(k/\epsilon)/\epsilon)$ iterations (Lines 5-13). It initiates two pair of disjoint sets: (X, Y) and (X', Y') (Line 4). At each iteration, it only updates alternatively partial solutions, either X, X' or Y, Y' , by calling ThreshSeq (Lines 7, 10). The algorithm then carefully selects candidates X'' and Y'' that consist of $\min\{k, |X'|\}$ and $\min\{k, |Y'|\}$ from X' and Y' , respectively (Lines 14-15). Finally, it returns the best among candidates without violating the cardinality constraint. At a high level, our BoostAdapt algorithm do not adapt USM algorithms (may make the approximation ratio worse) and operates differently from Iterated Greedy [Gupta *et al.*, 2010], which inspired AdaptiveThresholdGreedy of [Kuhnle, 2021a; Chen and Kuhnle, 2024]. BoostAdapt is an elegant combination between threshold greedy [Badanidiyuru and Vondrák, 2014] and twin greedy [Han *et al.*, 2020] via using ThreshSeq procedures alternatively to update two disjoint solutions X and Y . Interestingly, we found that X and Y can support each other to bound elements in the optimal solution. Besides, by carefully analyzing the role of the set X after the first iteration (i.e., X_1), we can give a better ratio than previous algorithms with the same complexities.

Before analyzing the BoostAdapt's performance, we provide some useful notations as follows:

- X^i, Y^i are the sets of first i elements added into X and Y , respectively.
- X_i and Y_i are X and Y after the iteration i of the main loop

Algorithm 3: BoostAdapt

```

1: Input:  $f, V, k, \epsilon$ .
2:  $S_0 \leftarrow \text{LinAdapt}(f, V, k, 1, \epsilon, 1/3)$ ,
    $a \leftarrow 12 + (\frac{16}{1-\epsilon} + \frac{8(2-\epsilon)}{(1-\epsilon)(1-2\epsilon)})\epsilon$ 
3:  $M \leftarrow af(S_0)$ ,  $\Delta \leftarrow \lceil \log_{\frac{1}{1-\epsilon}}(\frac{ak}{4\epsilon}) \rceil + 1$ ,  $\delta \leftarrow 1/(3\Delta)$ 
4:  $X, X', Y, Y' \leftarrow \emptyset$ ,  $k' \leftarrow \max\{i \in \mathbb{N} : (1-\epsilon)i \leq k\}$ ,
    $\tau \leftarrow \frac{k'M}{4k}$ 
5: for  $i \leftarrow 1$  to  $\Delta$  do
6:   if  $i$  is odd then
7:      $\tau_X \leftarrow \tau(1-\epsilon)^i$ ,  $(A_i, A'_i) \leftarrow$ 
        $\text{ThreshSeq}(f(X \cup \cdot), V \setminus (X \cup Y), k' - |X|, \epsilon, \delta, \tau_X)$ 
8:      $X \leftarrow X \cup A_i$ ,  $X' \leftarrow X' \cup A'_i$ 
9:   else
10:     $\tau_Y \leftarrow \tau(1-\epsilon)^i$ ,  $(B_i, B'_i) \leftarrow$ 
       $\text{ThreshSeq}(f(Y \cup \cdot), V \setminus (X \cup Y), k' - |Y|, \epsilon, \delta, \tau_Y)$ 
11:     $Y \leftarrow Y \cup B_i$ ,  $Y' \leftarrow Y' \cup B'_i$ 
12:   end if
13: end for
14: Define  $X' = \{x'_1, x'_2, \dots, x'_{|X'|}\}$ ,
    $Y' = \{y'_1, y'_2, \dots, y'_{|Y'|}\}$  contain elements in the order
   selected.
15:  $X'' \leftarrow$  set of  $\min\{k, |X'|\}$  elements  $x'_i \in X'$  with
   largest marginal gain  $f(x'_i | X'^{<x'_i})$ 
16:  $Y'' \leftarrow$  set of  $\min\{k, |Y'|\}$  elements  $y'_i \in Y'$  with largest
   marginal gains  $f(y'_i | Y'^{<y'_i})$ 
17:  $S \leftarrow \arg \max_{Z \in \{X', Y', X'', Y'', S_0\}, |Z| \leq k} f(Z)$ 
18: Return  $S$ 

```

and $X_0 = Y_0 = \emptyset$; X'_i and Y'_i are defined analogously.
- A_i^+ (res. B_i^+) as the set of elements in A_i (res. B_i) having the marginal gain at least τ_X (res. τ_Y) at the iteration i , $X_i^+ = \bigcup_{j=1}^i A_j^+$, $Y_i^+ = \bigcup_{j=1}^i B_j^+$ and $X^+ = X_\Delta^+$, $Y^+ = Y_\Delta^+$. By the properties of ThreshSeq [Chen and Kuhnle, 2024], it is noted that $|A_i^+| \geq (1-\epsilon)|A_i|$, $|B_i^+| \geq (1-\epsilon)|B_i|$.
- For an element $e \in X \cup Y$, we denote by $X^{<e}$, $Y^{<e}$ as X , Y right before e is selected into X or Y , respectively.

We now analyze theoretical bounds of BoostAdapt algorithm. Firstly, we explore some basic properties of X, X', X'' and Y, Y', Y'' according to the iteration i of the main loop.

Lemma 3. *a) For any iteration i of the outer loop (Lines 5-13) of Algorithm 3, we have $f(X'_i) \geq f(X_i)$ and $f(Y'_i) \geq f(Y_i)$. b) At the end of BoostAdapt, we have $f(X'') \geq (1-\epsilon)f(X')$ and $f(Y'') \geq (1-\epsilon)f(Y')$.*

By carefully analyzing the relationship between X and Y through thresholds, we provide the connection between X and Y after each iteration in Lemma 4.

Lemma 4. *If BoostAdapt succeeds and $X_1 = \emptyset$, after iteration i the following properties hold:*

(a) *If i is odd and $|X_i| < k'$: for any sets $C \subseteq Y_{i-1}^+$ and $C' \subseteq Y_{i-1} \setminus X_{i-1}^+$, we have $\sum_{x \in C} f(x | X_i) \leq \sum_{x \in C'} \frac{f(e | Y^{<e})}{1-\epsilon}$ and $\sum_{e \in C'} f(e | X_i) = \frac{\epsilon}{1-\epsilon} f(Y_{i-1}^+)$.*

(b) *If i is even and $|Y_i| < k'$, for any set $D \subseteq X_{i-1}^+$ and $D' \subseteq X_{i-1} \setminus X_{i-1}^+$, we have $\sum_{x \in D} f(x | Y_i) \leq \sum_{x \in D'} \frac{f(e | X^{<e})}{1-\epsilon}$ and $\sum_{e \in D'} f(e | Y_i) = \frac{\epsilon}{1-\epsilon} f(X_{i-1}^+)$.*

Using Lemma 4 and the fact that if $|T| = k$, then $|T \setminus O| \geq |O \setminus T|$, where $T \in \{X^+, Y^+\}$, we can bound of the optimal solution in some cases related to the size of X and Y in Lemma 5.

Lemma 5. *If BoostAdapt succeeds and $X_1 = \emptyset$, at the end of BoostAdapt we have*

a) *If $|X| < k'$ and $|Y| < k'$, then $f(O) < \frac{4f(S)}{(1-\epsilon)^2(1-2\epsilon)}$.*
b) *If there exists $T \in \{X, Y\}$ such that $|T| = k'$, then $f(S) \geq (1-\epsilon)^4 \frac{\text{opt}}{4}$.*

Finally, putting them together, we give the tighter ratio in Theorem 6.

Theorem 6. *For any $\epsilon \in (0, 1/4)$ the algorithm runs in $O(\log(n/\epsilon) \log(k/\epsilon)/\epsilon^2)$ adaptive complexity, $O(n \log(k/\epsilon)/\epsilon^2)$ query complexity in expectation. The algorithm succeeds with a probability of at least $1 - 1/n$. If the algorithm succeeds, it returns an approximation ratio of $1/4 - \epsilon$.*

Proof. Due to the space limit, we put proof for successful probability and complexities in the appendix. If the algorithm succeeds, we now prove the ratio by considering the following cases: (1) If $X_1 \neq \emptyset$ or $Y_2 \neq \emptyset$, we have $f(S) \geq \max\{f(X_1), f(Y_2)\} \geq \frac{(1-\epsilon)^2 k' M}{4k} \geq (1-\epsilon)^2 \frac{\text{opt}}{4} > (\frac{1}{4} - \epsilon) \text{opt}$. (2) If X_1 and Y_2 are both empty. If $|X| < k'$ and $|Y| < k'$, from Lemma 5, we have $f(S) > \frac{(1-\epsilon)^2(1-2\epsilon)}{4} \text{opt} > (\frac{1}{4} - \epsilon) \text{opt}$. If there exists $T \in \{X, Y\}$ such that $|T| = k$, then $f(S) \geq (1-\epsilon)^4 \frac{\text{opt}}{4} > (\frac{1}{4} - \epsilon) \text{opt}$. This completes the proof. $\square$

7 Empirical Study

In this section, we compare our LinAst, LinAtg and BoostAdapt¹ with state-of-the-art for non-monotone SMC, including IteratedGreedy (IG) [Gupta et al., 2010], FastRandomGreedy (FRG) [Buchbinder et al., 2015], PITG [Chen et al., 2025], ParSSP [Cui et al., 2023], FASTLS+GUIDEDRG (FLS) [Chen et al., 2024a], AdaptiveSimpleThreshold (AST) [Chen and Kuhnle, 2024], and AdaptiveThreshold-Greedy (ATG) [Chen and Kuhnle, 2024]. The comparison is about **four metrics**: object values, adaptive complexity, number of queries, and running time. We experimented with two applications: Revenue Maximization (RM) and Maximum Cut (MC) [Chen and Kuhnle, 2024; Kuhnle, 2019; Amanatidis et al., 2020]. **Dataset and Setting.** For all algorithms, we've fixed the parameter $\epsilon = 0.1$. For our proposed algorithms, the parameter α was set to 4.0. Each algorithm was executed over 20 independent runs, and the reported results are the average over these runs. We used Astro ($n = 18,772, m = 198,110$) for RM and utilized web-Google ($n = 875,713, m = 5,105,039$), GrQc ($n = 5,242, m =$

¹The source code of our algorithms is available at <https://github.com/tantdhvan/AdaptIJCAI26>

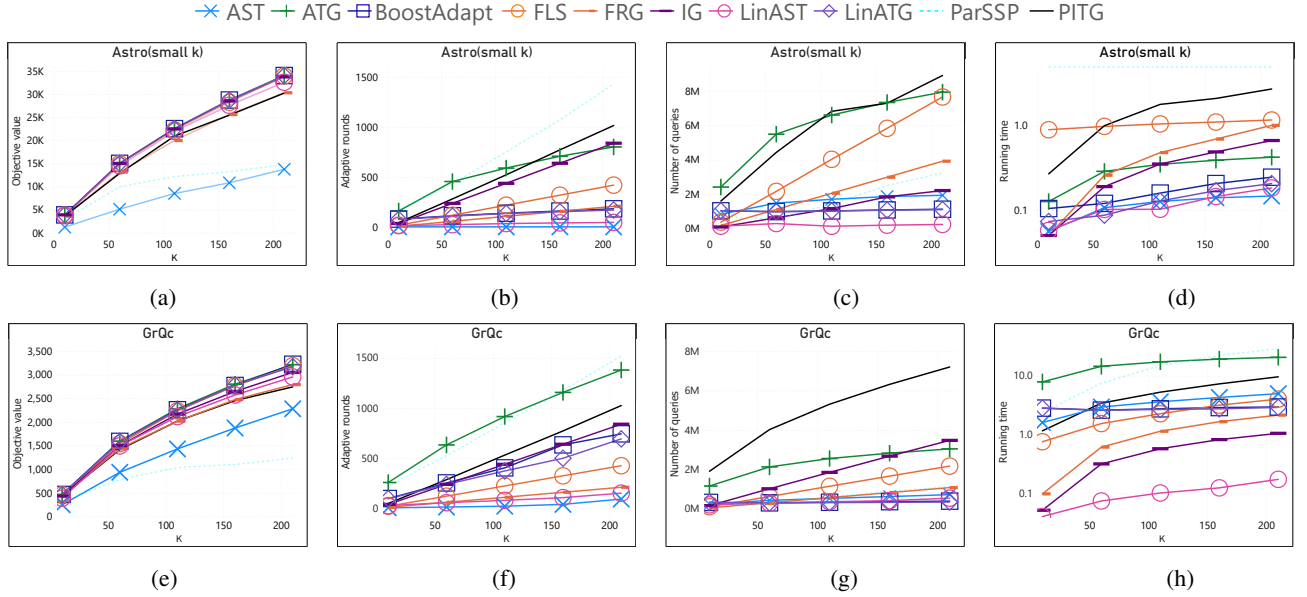


Figure 1: Performance of algorithms on Revenue Maximization (a-d) and Maximum Cut (e-h).

14, 496), and Barabasi-Albert ($n = 968, m = 5, 708$) for MC. **More detailed settings** given in the appendix.

Figure 1 displays the results on the Astro and GrQc datasets. Additional results are presented in the appendix.

Objective value. Figures 1(a)(e) show the objective values of algorithms. It can be observed that the lines of BoostAdapt on the top while they consistently reach the highest points with every k . The top includes FLS, BoostAdapt, LinAtg, IG and AST. Both LinAst, PITG and FRG mark a little lower than BoostAdapt, followed by AST, and ParSSP. However, with small k , the gap between lines on the top seem inconsiderable. Finally, our algorithms significantly contribute to the quality of the solution. **Adaptive rounds.** In the MC application (Fig. 1(f)), the adaptive rounds result three distinct groups. The group with low adaptivity includes AST, LinAst, and FRG while the medium group includes IG, LinAtg, FLS, PITG and BoostAdapt. The high group is the remaining. ParSSP and ATG algorithms waste the highest number of adaptive rounds while AST hits the lowest points. With the RM instance (Fig. 1(b)), ParSSP, ATG, PITG, and IG give high numbers of adaptivities. When k increases, their quantities of the adaptive rounds seem to grow quickly, 4-10 times larger than the others. LinAtg slightly differs from the lowest AST while LinAst and BoostAdapt are higher than AST, but they do not exceed 200. **Number of queries.** In both RM and MC (Figures 1(c) and (g)), our algorithms almost always minimize the number of queries. In RM (Figure 1(c)), LinAst has the lowest number of queries, followed by LinAtg and BoostAdapt algorithms. The number of queries for AST is slightly higher than that of BoostAdapt. The remaining with ATG, FRG, ParSSP, FLS, PITG, and IG belong to the high-query number group. Especially, ATG and PITG mark the highest. For MC (Figure 1(g)), LinAtg reaches the lowest, followed by BoostAdapt and LinAst. The algorithms AST, ParSSP, and FRG again perform at an average level. In contrast, PITG typically requires the highest num-

ber of queries-approximately five times more than the lowest, LinAtg. On the whole, all our algorithms save queries more than the others. Moreover, our query numbers remain steady over the range of k . This is consistent with (nearly) linear query complexities of our algorithms. **Time taken.** In MC (Figure 1(h)), LinAst wastes the lowest time, followed by IG and FRG. While BoostAdapt, FLS, and LinAtg have the same running time, which is higher than IG and FRG but lower than AST, PITG, and ParSSP. The gap among them considerably reduces, especially when k increases, except ParSSP. Meanwhile, the time consumption of ParSSP steadily grows up along with k 's growth. In RM (Figure 1(d)), LinAst and AST have the lowest running time, closely followed by the group of LinAtg, BoostAdapt, ATG. The rest spend more time than the others, especially ParSSP, which runs the slowest, about 10-20 times higher than the others. **Overall,** these metrics demonstrate that our algorithms consistently achieve higher-quality solutions while requiring substantially fewer queries and a small number of adaptive rounds. In particular, compared to PITG, the most recent state-of-the-art method, our proposed BoostAdapt algorithm attains better objective values with significantly lower iteration counts and query complexities. For the RM application with $k = 210$, BoostAdapt reduces both parallel complexity and query complexity by approximately 5-7 times.

8 Conclusions

Motivated by the big data challenge for non-monotone submodular maximization under cardinality constraint, we focus on parallel approximation algorithms based on the concept of adaptive complexity. In particular, we proposed efficient parallel algorithms that significantly improve both approximation ratio and query complexity but keep the near-optimal adaptive complexity of $O(\log n)$. Our algorithm also expresses superior solution quality and computation complexity compared to state-of-the-art algorithms.

Acknowledgement

Tan D. Tran was funded by the Master, PhD Scholarship Programme of Vingroup Innovation Foundation (VINIF), code VINIF.2024.TS.069.

References

- [Amanatidis *et al.*, 2020] Georgios Amanatidis, Federico Fusco, Philip Lazos, Stefano Leonardi, and Rebecca Reiffenhäuser. Fast adaptive non-monotone submodular maximization subject to a knapsack constraint. In *Annual Conference on Neural Information Processing Systems*, 2020.
- [Amanatidis *et al.*, 2021] Georgios Amanatidis, Federico Fusco, Philip Lazos, Stefano Leonardi, Alberto Marchetti-Spaccamela, and Rebecca Reiffenhäuser. Submodular maximization subject to a knapsack constraint: Combinatorial algorithms with near-optimal adaptive complexity. In *International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 231–242, 2021.
- [Badanidiyuru and Vondrák, 2014] Ashwinkumar Badanidiyuru and Jan Vondrák. Fast algorithms for maximizing submodular functions. In *Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1497–1514, 2014.
- [Balkanski and Singer, 2018] Eric Balkanski and Yaron Singer. The adaptive complexity of maximizing a submodular function. In *Annual ACM SIGACT Symposium on Theory of Computing*, pages 1138–1151, 2018.
- [Balkanski *et al.*, 2018] Eric Balkanski, Adam Breuer, and Yaron Singer. Non-monotone submodular maximization in exponentially fewer iterations. In Samy Bengio, Hanna M. Wallach, Hugo Larochelle, Kristen Grauman, Nicolò Cesa-Bianchi, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3-8, 2018, Montréal, Canada*, pages 2359–2370, 2018.
- [Balkanski *et al.*, 2019] Eric Balkanski, Aviad Rubinfeld, and Yaron Singer. An exponential speedup in parallel running time for submodular maximization without loss in approximation. In *Proceedings of the Thirtieth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2019, San Diego, California, USA, January 6-9, 2019*, pages 283–302. SIAM, 2019.
- [Breuer *et al.*, 2020] Adam Breuer, Eric Balkanski, and Yaron Singer. The FAST algorithm for submodular maximization. In *Proc. of the International Conference on Machine Learning*, volume 119, pages 1134–1143, 2020.
- [Buchbinder and Feldman, 2024] Niv Buchbinder and Moran Feldman. Constrained submodular maximization via new bounds for dr-submodular functions. In *Proceedings of the 56th Annual ACM Symposium on Theory of Computing, STOC 2024, Vancouver, BC, Canada, June 24-28, 2024*, pages 1820–1831. ACM, 2024.
- [Buchbinder *et al.*, 2012] Niv Buchbinder, Moran Feldman, Joseph Naor, and Roy Schwartz. A tight linear time $(1/2)$ -approximation for unconstrained submodular maximization. In *IEEE Symposium on Foundations of Computer Science*, pages 649–658, 2012.
- [Buchbinder *et al.*, 2014] Niv Buchbinder, Moran Feldman, Joseph Naor, and Roy Schwartz. Submodular maximization with cardinality constraints. In *Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1433–1452. SIAM, 2014.
- [Buchbinder *et al.*, 2015] Niv Buchbinder, Moran Feldman, and Roy Schwartz. Comparing apples and oranges: Query tradeoff in submodular maximization. In *Proc. of the 26th ACM-SIAM SODA 2015*, pages 1149–1168, 2015.
- [Chekuri and Quanrud, 2019] Chandra Chekuri and Kent Quanrud. Parallelizing greedy for submodular set function maximization in matroids and beyond. In Moses Charikar and Edith Cohen, editors, *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing, STOC 2019, Phoenix, AZ, USA, June 23-26, 2019*, pages 78–89. ACM, 2019.
- [Chen and Kuhnle, 2023] Yixin Chen and Alan Kuhnle. Approximation algorithms for size-constrained non-monotone submodular maximization in deterministic linear time. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD 2023, Long Beach, CA, USA, August 6-10, 2023*, pages 250–261. ACM, 2023.
- [Chen and Kuhnle, 2024] Yixin Chen and Alan Kuhnle. Practical and parallelizable algorithms for non-monotone submodular maximization with size constraint. *Journal of Artificial Intelligence Research*, 79:599–637, 2024.
- [Chen *et al.*, 2019] Lin Chen, Moran Feldman, and Amin Karbasi. Unconstrained submodular maximization with constant adaptive complexity. In *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing, STOC 2019*, page 102–113, 2019.
- [Chen *et al.*, 2021] Yixin Chen, Tonmoy Dey, and Alan Kuhnle. Best of both worlds: Practical and theoretically optimal submodular maximization in parallel. In Marc’Aurelio Ranzato, Alina Beygelzimer, Yann N. Dauphin, Percy Liang, and Jennifer Wortman Vaughan, editors, *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual*, pages 25528–25539, 2021.
- [Chen *et al.*, 2024a] Yixin Chen, Ankur Nath, Chunli Peng, and Alan Kuhnle. Discretely beyond $1/e$: Guided combinatorial algorithms for submodular maximization. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- [Chen *et al.*, 2024b] Yixin Chen, Ankur Nath, Chunli Peng, and Alan Kuhnle. Discretely beyond $1/e$: Guided combinatorial algorithms for submodular maximization. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- [Chen *et al.*, 2025] Yixin Chen, Wenjing Chen, and Alan Kuhnle. Breaking barriers: Combinatorial algorithms for

- non-monotone submodular maximization with sublinear adaptivity and $1/e$ approximation. *CoRR*, abs/2502.07062, 2025.
- [Cui *et al.*, 2023] Shuang Cui, Kai Han, Jing Tang, He Huang, Xueying Li, and Aakas Zhiyuli. Practical parallel algorithms for submodular maximization subject to a knapsack constraint with nearly optimal adaptivity. In *Thirty-Seventh AAAI Conference on Artificial Intelligence, AAAI 2023, Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence, IAAI 2023, Thirteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2023, Washington, DC, USA, February 7-14, 2023*, pages 7261–7269. AAAI Press, 2023.
- [Ene and Nguyen, 2020] Alina Ene and Huy L. Nguyen. Parallel algorithm for non-monotone dr-submodular maximization. In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*, volume 119 of *Proceedings of Machine Learning Research*, pages 2902–2911. PMLR, 2020.
- [Ene *et al.*, 2019] Alina Ene, Huy L. Nguyen, and Adrian Vladu. Submodular maximization with matroid and packing constraints in parallel. In *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing, STOC 2019, Phoenix, AZ, USA, June 23-26, 2019*, pages 90–101. ACM, 2019.
- [Fahrbach *et al.*, 2019a] Matthew Fahrbach, Vahab S. Mirrokni, and Morteza Zadimoghaddam. Non-monotone submodular maximization with nearly optimal adaptivity and query complexity. In Kamalika Chaudhuri and Ruslan Salakhutdinov, editors, *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA*, volume 97 of *Proceedings of Machine Learning Research*, pages 1833–1842. PMLR, 2019.
- [Fahrbach *et al.*, 2019b] Matthew Fahrbach, Vahab S. Mirrokni, and Morteza Zadimoghaddam. Submodular maximization with nearly optimal approximation, adaptivity and query complexity. In *Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 255–273, 2019.
- [Fahrbach *et al.*, 2023] Matthew Fahrbach, Vahab Mirrokni, and Morteza Zadimoghaddam. Non-monotone submodular maximization with nearly optimal adaptivity and query complexity. *preprint, arXiv:1808.06932*, 2023.
- [Gupta *et al.*, 2010] Anupam Gupta, Aaron Roth, Grant Schoenebeck, and Kunal Talwar. Constrained non-monotone submodular maximization: Offline and secretary algorithms. In *International Workshop on Internet and Network Economics*, 2010.
- [Han *et al.*, 2020] Kai Han, Zongmai Cao, Shuang Cui, and Benwei Wu. Deterministic approximation for submodular maximization over a matroid in nearly linear time. In Hugo Larochelle, Marc’Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin, editors, *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020.
- [Han *et al.*, 2021] Kai Han, Shuang Cui, Tianshuai Zhu, Enpei Zhang, Benwei Wu, Zhizhuo Yin, Tong Xu, Shaojie Tang, and He Huang. Approximation algorithms for submodular data summarization with a knapsack constraint. *Proc. ACM Meas. Anal. Comput. Syst.*, 5(1):05:1–05:31, 2021.
- [Hartline *et al.*, 2008] Jason D. Hartline, Vahab S. Mirrokni, and Mukund Sundararajan. Optimal marketing strategies over social networks. In *Proceedings of the 17th International Conference on World Wide Web, WWW 2008, Beijing, China, April 21-25, 2008*, pages 189–198. ACM, 2008.
- [Kazemi *et al.*, 2019] Ehsan Kazemi, Marko Mitrovic, Morteza Zadimoghaddam, Silvio Lattanzi, and Amin Karbasi. Submodular streaming in all its glory: Tight approximation, minimum memory and low adaptive complexity. In *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA*, volume 97 of *Proceedings of Machine Learning Research*, pages 3311–3320. PMLR, 2019.
- [Kuhnle, 2019] Alan Kuhnle. Interlaced greedy algorithm for maximization of submodular functions in nearly linear time. In *Neural Information Processing Systems*, pages 2371–2381, 2019.
- [Kuhnle, 2021a] Alan Kuhnle. Nearly linear-time, parallelizable algorithms for non-monotone submodular maximization. In *Proc. of the 30th AAAI 2021*, pages 8200–8208, 2021.
- [Kuhnle, 2021b] Alan Kuhnle. Quick streaming algorithms for maximization of monotone submodular functions in linear time. In *Proc. of the 24th AISTATS 2021*, volume 130 of *Proc. of MLR*, pages 1360–1368, 2021.
- [Lee *et al.*, 2010] Jon Lee, Vahab S. Mirrokni, Viswanath Nagarajan, and Maxim Sviridenko. Maximizing non-monotone submodular functions under matroid or knapsack constraints. *SIAM J. Discret. Math.*, 23(4):2053–2078, 2010.
- [Leskovec *et al.*, 2007] Jure Leskovec, Jon Kleinberg, and Christos Faloutsos. Graph evolution: Densification and shrinking diameters. *ACM transactions on Knowledge Discovery from Data (TKDD)*, 1(1):2–es, 2007.
- [Leskovec *et al.*, 2009] Jure Leskovec, Kevin J Lang, Anirban Dasgupta, and Michael W Mahoney. Community structure in large networks: Natural cluster sizes and the absence of large well-defined clusters. *Internet Mathematics*, 6(1):29–123, 2009.
- [Li *et al.*, 2022] Wenxin Li, Moran Feldman, Ehsan Kazemi, and Amin Karbasi. Submodular maximization in clean linear time. In *Advances in Neural Information Processing Systems*, pages 7887–7897, 2022.
- [Lin and Bilmes, 2011] Hui Lin and Jeff A. Bilmes. A class of submodular functions for document summarization. In *Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, pages 510–520, 2011.

- [Mirzasoleiman *et al.*, 2016] Baharan Mirzasoleiman, Ashwinkumar Badanidiyuru, and Amin Karbasi. Fast constrained submodular maximization: Personalized data summarization. In *International Conference on Machine Learning*, volume 48 of *JMLR Workshop and Conf. Proc.*, pages 1358–1367, 2016.
- [Mitzenmacher and Upfal, 2005] Michael Mitzenmacher and Eli Upfal. *Probability and Computing: Randomized Algorithms and Probabilistic Analysis*. Cambridge University Press, 2005.
- [Nemhauser *et al.*, 1978] George L. Nemhauser, Laurence A. Wolsey, and Marshall L. Fisher. An analysis of approximations for maximizing submodular set functions - I. *Math. Program.*, 14(1):265–294, 1978.
- [Tukan *et al.*, 2024] Murad Tukan, Loay Muallem, and Moran Feldman. Practical 0.385-approximation for submodular maximization subject to a cardinality constraint. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- [Wald, 1945] Abraham Wald. Some generalizations of the theory of cumulative sums of random variables. *The Annals of Mathematical Statistics*, 16(3):287–293, 1945.

Organization of the Appendix

- Section A presents simplified version of BoostAdapt algorithm and discussion.
- Section B presents some essential Probability Lemmas and Concentration Bounds used in this work.
- Section C presents algorithms missed in Section 3.
- Section D presents the Algorithms and proofs missed of Section 4.
- Section F presents the proofs missed of Section 6.
- Section G presents the additional experimental details and results of Section 7.
- Section H provides additional related work for monotone SMC.

A Simplified Version of BoostAdapt algorithm

To easily follow the idea of the BoostAdapt algorithm, this section presents a simplified version of Boost and some discussion on related algorithms.

This version operates in $O(\log_{\frac{1}{1-\epsilon}}(\frac{k^2}{4\epsilon}))$ iterations. It adapts greedy threshold [Badanidiyuru and Vondrák, 2014] to update two disjoint sets X and Y . In particular, it alternately selects elements with the marginal gain at least τ_X (τ_Y) into X (Y). Note that the thresholds τ_X and τ_Y are updated alternately after each iteration. Finally, the algorithm returns the best solution among X and Y .

It is noted that our algorithm operates in a different way from Iterated Greedy [Gupta *et al.*, 2010], which works as follows: (1) it repeats greedy methods to construct two solutions: finding a feasible solution X over ground set V then finding another solution Y over new ground set $V \setminus X$; (2) it then adapts the unconstrained submodular maximization (USM) algorithm to find the solution X' over ground set X and returns the best one among X , Y , and X' . Our algorithm also differs from the twin greedy strategy [Han *et al.*, 2020] and Interlaced Greedy [Kuhnle, 2019], allowing simultaneously updating two disjoint sets simultaneously in each iteration. In contrast to these algorithms, our algorithm allows the integration of ThreshSeq in each iteration; thus, it can be effectively parallelized.

Algorithm 4: Simplified Version of BoostAdapt Algorithm

```

1: Input:  $f, V, k, \epsilon$ 
2:  $M \leftarrow \max_{e \in V} f(e)$ 
3:  $\Delta \leftarrow \lceil \log_{\frac{1}{1-\epsilon}}(\frac{k^2}{4\epsilon}) \rceil + 1, \tau \leftarrow kM/4$ 
4:  $X \leftarrow \emptyset, Y \leftarrow \emptyset$ 
5: for  $i \leftarrow 1$  to  $\Delta$  do
6:   if  $i$  is odd then
7:      $\tau_X \leftarrow \tau(1 - \epsilon)^i$ 
8:     for  $e \in V \setminus (X \cup Y)$  do
9:       if  $f(e|X) \geq \tau_X$  then
10:         $X \leftarrow X \cup \{e\}$ 
11:       end if
12:     end for
13:   else
14:      $\tau_Y \leftarrow \tau(1 - \epsilon)^i$ 
15:     for  $e \in V \setminus (X \cup Y)$  do
16:       if  $f(e|Y) \geq \tau_Y$  then
17:         $Y \leftarrow Y \cup \{e\}$ 
18:       end if
19:     end for
20:   end if
21: end for
22:  $S \leftarrow \arg \max_{Z \in \{X, Y\}} f(Z)$ 
23: Return  $S$ 

```

We define the following notations used for analyzing the algorithm's performance guarantees

- Supposing X and Y ordered: $X = \{x_1, x_2, \dots, x_{|X|}\}, Y = \{y_1, y_2, \dots, y_{|Y|}\}$, we conduct: $X^i = \{x_1, x_2, \dots, x_i\}, Y^i = \{y_1, y_2, \dots, y_i\}$, where x_i (or y_i) is i -th element added into X (or Y).
- For $e \in X \cup Y$, we assume that x is added into X or Y at iteration $l(e)$.

- X_i and Y_i are X and Y after the iteration i of the main loop and $X_0 = Y_0 = \emptyset$.
- For an element $e \in X \cup Y$, we denote $X^{<e}, Y^{<e}, \tau_X^e$ and τ_Y^e are X, Y, τ_X and τ_Y right before e is selected into X or Y , respectively.
- τ_X^{last} and τ_Y^{last} are τ_X and τ_Y at the last iterations when X and Y are considered to update.

We provide the performance guarantees of Algorithm 4 in Theorem 7.

Theorem 7. For any $\epsilon \in (0, 1/4)$, the Algorithm 4 takes $O(\frac{n}{\epsilon} \log(\frac{k}{\epsilon}))$ query complexity and return an approximation ratio of $1/4 - \epsilon$.

Proof. The Algorithm 4 takes n queries to find M and then runs in $\Delta = O(\log_{1/(1-\epsilon)}(\frac{k^2}{4\epsilon})) = O(\frac{1}{\epsilon} \log(\frac{k}{\epsilon}))$ iterations. Each iteration takes $O(n)$ queries to update X or Y . Thus, the algorithm takes total $O(n) + O(\frac{n}{\epsilon} \log(\frac{k}{\epsilon})) = O(\frac{n}{\epsilon} \log(\frac{k}{\epsilon}))$.

For the approximation ratio. If $|X| \neq \emptyset$ or $|Y| \neq \emptyset$, i.e., it contains at least one element, we have

$$\begin{aligned} f(S) &\geq \frac{(1-\epsilon)^2}{4} \max\{f(X_1), f(Y_2)\} \geq \frac{(1-\epsilon)^2 kM}{4} \\ &\geq \frac{(1-\epsilon)^2 \text{opt}}{4} > (\frac{1}{4} - \epsilon) \text{opt}. \end{aligned}$$

We consider the other case when $X_1 = Y_2 = \emptyset$. In this case $f(X_1) = 0$, so $f(X_1 \cap O) = 0$. We further consider the following sub-cases

- If $|X| < k$ and $|Y| < k$. Considering an element e is added into Y (at iteration $l(e) \geq 4$), it was not added into X at previous iterations. Therefore, it has marginal gain $f(e|X^{<e}) < \theta_X^e = \frac{\theta_Y^{l(e)}}{1-\epsilon} \leq \frac{f(e|Y^{<e})}{1-\epsilon}$. Combining this with $\frac{kM}{4} \geq \tau \geq \frac{(1-\epsilon)\epsilon M}{k}$, we have

$$f(O \cup X) - f(X) \leq \sum_{e \in O \setminus X} f(e|X) \quad (5)$$

$$= \sum_{e \in O \cap Y} f(e|X) + \sum_{e \in O \setminus (X \cup Y)} f(e|X) \quad (6)$$

$$\leq \sum_{e \in O \cap Y} \frac{f(e|Y^{<e})}{1-\epsilon} + \tau_X^{last} k \quad (7)$$

$$< \frac{f(Y)}{1-\epsilon} + \epsilon \text{opt} \quad (8)$$

where inequality (5) is due to the submodularity of f , the inequality (7) is due to: $\tau_X^{last} \leq \epsilon M/k \leq \epsilon \text{opt}/k$, $f(Y) = \sum_{e \in Y} f(e|Y^{<e})$ and Y contains elements with positive marginal gain.

On the other hand, since $X_1 = \emptyset$ any element e is added into X (at iteration $l(e) \geq 2$) having marginal gain $f(e|Y^{<e}) < \theta_Y^e = \theta_X^{l(e)-1} = \frac{\theta_Y^{l(e)}}{1-\epsilon} \leq \frac{f(e|X^{<e})}{1-\epsilon}$. Thus

$$f(O \cup Y) - f(Y) \leq \sum_{e \in O \cap X} f(e|Y) + \sum_{e \in O \setminus (X \cup Y)} f(e|X) \quad (9)$$

$$\leq \sum_{e \in O \cap X} \frac{f(e|X^{<e})}{1-\epsilon} + \tau_Y^{last} k \quad (10)$$

$$< \frac{f(X)}{1-\epsilon} + \epsilon \text{opt}. \quad (11)$$

By combining the submodularity of f , inequalities (8) and (11) and the selection rule of the final solution, we have:

$$\begin{aligned} f(O) &\leq f(O \cup X) + f(O \cup Y) \\ &\leq \frac{f(X) + f(Y)}{1-\epsilon} + f(X) + f(Y) + 2\epsilon \text{opt} \\ &\leq 2 \frac{2-\epsilon}{1-\epsilon} f(S) + 2\epsilon \text{opt} \end{aligned}$$

which implies that

$$f(S) \geq \frac{(1-2\epsilon)(1-\epsilon)}{2(2-\epsilon)} \text{opt} > (\frac{1}{2} - \epsilon)(\frac{1}{2} - \frac{\epsilon}{2}) > (\frac{1}{4} - \epsilon) \text{opt}.$$

- If $|X| = k$ and $|Y| = k$, then $l(x_k) \geq 3$. Any element e is not added into X at iteration $l(x_k)$ is not added into X at iteration $l(x_k) - 2$. On the other hand, since $|X| = k \geq |O|$, so $|X \setminus O| \geq |O \setminus X|$, we have:

$$\begin{aligned}
f(X \cup O) - f(X) &\leq \sum_{e \in O \setminus X} f(e|X) \\
&\leq \sum_{e \in O \cap Y} f(e|X) + \sum_{e \in O \setminus (X \cup Y)} f(e|X) \\
&< \frac{\sum_{e \in O \cap Y} f(e|Y^{<e})}{1 - \epsilon} + |O \setminus X| \cdot \tau_X^{l(x_k)-2} \\
&\leq \frac{\sum_{e \in O \cap Y} f(e|Y^{<e})}{1 - \epsilon} + |X \setminus O| \cdot \frac{\tau_X^{l(x_k)}}{(1 - \epsilon)^2} \\
&\leq \frac{\sum_{e \in O \cap Y} f(e|Y^{<e})}{1 - \epsilon} + \sum_{e \in X \setminus O} \frac{f(e|X^{<e})}{(1 - \epsilon)^2}.
\end{aligned}$$

Similarly, we have

$$f(O \cup Y) - f(Y) \leq \sum_{e \in O \cap X} f(e|Y) + \sum_{e \in O \setminus (Y \cup X)} f(e|X) \quad (12)$$

$$< \frac{\sum_{e \in O \cap X} f(e|X^{<e})}{1 - \epsilon} + \tau_Y^{l(y_k)-2} k \quad (13)$$

$$= \frac{\sum_{e \in O \cap X} f(e|X^{<e})}{1 - \epsilon} + \frac{\tau_Y^{l(y_k)}}{(1 - \epsilon)^2} k \quad (14)$$

$$\leq \frac{\sum_{e \in O \cap X} f(e|X^{<e})}{1 - \epsilon} + \sum_{e \in Y \setminus O} \frac{f(e|Y^{<e})}{(1 - \epsilon)^2}. \quad (15)$$

By combining (12) and (15), we have

$$\begin{aligned}
f(O) &< \frac{\sum_{e \in O \cap Y} f(e|Y^{<e})}{1 - \epsilon} + \sum_{e \in Y \setminus O} \frac{f(e|Y^{<e})}{(1 - \epsilon)^2} + \sum_{e \in X \setminus O} \frac{f(e|X^{<e})}{(1 - \epsilon)^2} + \frac{\sum_{e \in O \cap X} f(e|X^{<e})}{1 - \epsilon} + f(X) + f(Y) \\
&< \frac{f(X) + f(Y)}{(1 - \epsilon)^2} + 2f(S) \\
&= (2 + \frac{2}{(1 - \epsilon)^2})f(S) < \frac{4}{(1 - \epsilon)^2}f(S)
\end{aligned}$$

which implies that $f(S) \geq \frac{(1-\epsilon)^2}{4} \text{opt} > (\frac{1}{4} - \epsilon) \text{opt}$.

- If $|X| < k$ and $|Y| = k$, by applying the transformations to (7) and (15) we have

$$f(O \cup X) - f(X) \leq \sum_{e \in O \cap Y} \frac{f(e|Y^{<e})}{1 - \epsilon} + \epsilon \text{opt}$$

and

$$f(O \cup Y) - f(Y) \leq \frac{\sum_{e \in O \cap X} f(e|X^{<e})}{1 - \epsilon} + \sum_{e \in Y \setminus O} \frac{f(e|Y^{<e})}{(1 - \epsilon)^2}.$$

Therefore

$$\begin{aligned}
f(O) &\leq f(O \cup X) + f(O \cup Y) \\
&< \sum_{e \in O \cap Y} \frac{f(e|Y^{<e})}{1-\epsilon} + \sum_{e \in Y \setminus O} \frac{f(e|Y^{<e})}{(1-\epsilon)^2} \\
&\quad + \frac{\sum_{e \in O \cap X} f(e|X^{<e})}{1-\epsilon} + f(X) + f(Y) + \epsilon \text{opt} \\
&\leq \frac{f(Y)}{(1-\epsilon)^2} + \frac{f(X)}{1-\epsilon} + 2f(S) + \epsilon \text{opt} \\
&< \frac{4}{(1-\epsilon)^2} f(S) + \epsilon \text{opt}.
\end{aligned}$$

Thus $f(S) \geq \frac{(1-\epsilon)^3}{4} \text{opt} > (\frac{1}{4} - \epsilon) \text{opt}$.

- If $|X| = k$ and $|Y| < k$, by applying the transformations to (10) and (12) we also have

$$f(O \cup Y) - f(Y) \leq \sum_{e \in O \cap X} \frac{f(e|X^{<e})}{1-\epsilon} + \epsilon \text{opt} \quad (16)$$

and

$$f(O \cup X) - f(X) \leq \frac{\sum_{e \in O \cap Y} f(e|Y^{<e})}{1-\epsilon} + \sum_{e \in X \setminus O} \frac{f(e|X^{<e})}{(1-\epsilon)^2}.$$

Therefore

$$\begin{aligned}
f(O) &\leq f(O \cup X) + f(O \cup Y) \\
&< \sum_{e \in O \cap X} \frac{f(e|X^{<e})}{1-\epsilon} + \sum_{e \in X \setminus O} \frac{f(e|X^{<e})}{(1-\epsilon)^2} \\
&\quad + \frac{\sum_{e \in O \cap Y} f(e|Y^{<e})}{1-\epsilon} + f(X) + f(Y) + \epsilon \text{opt} \\
&\leq \frac{f(X)}{(1-\epsilon)^2} + \frac{f(Y)}{1-\epsilon} + 2f(S) + \epsilon \text{opt} \\
&< \frac{4}{(1-\epsilon)^2} f(S) + \epsilon \text{opt}.
\end{aligned}$$

Thus $f(S) \geq \frac{(1-\epsilon)^3}{4} \text{opt} > (\frac{1}{4} - \epsilon) \text{opt}$.

Combining all cases, we complete the proof. $\square$

B Probability Lemmas and Concentration Bounds

This section provides Probability Lemmas and Concentration Bounds that are useful for analyzing the theoretical bounds of our algorithms.

Lemma 6 (Chernoff bounds [Mitzenmacher and Upfal, 2005]). *Supposing that $X_1, \dots, X_n$ are independent binary random variables such that $\Pr[X_i = 1] = p_i$. Let $\mu = \sum_{i=1}^n p_i$, and $X = \sum_{i=1}^n X_i$. Then for any $\delta \geq 0$, we have*

$$\Pr[X \geq (1 + \delta)\mu] \leq e^{\frac{-\delta^2 \mu}{2 + \delta}}. \quad (17)$$

For $0 \leq \delta \leq 1$, we have

$$\Pr[X \leq (1 - \delta)\mu] \leq e^{\frac{-\delta^2 \mu}{2}}. \quad (18)$$

Lemma 7 (Lemma 6 in [Chen et al., 2021]). *Suppose there is a sequence of n Bernoulli trials: $X_1, X_2, \dots, X_n$, where the success probability of X_i depends on the preceding trials $X_1, X_2, \dots, X_{i-1}$. Suppose it holds that*

$$\Pr[X_i = 1 | X_1 = x_1, X_2 = x_2, \dots, X_{i-1} = x_{i-1}] \geq \eta, \quad (19)$$

where $\eta > 0$ is a constant and $x_1, \dots, x_{i-1}$ are arbitrary.

Then if $Y_1, \dots, Y_n$ are independent Bernoulli trials, each with probability η of success, then

$$\Pr\left(\sum_{i=1}^n X_i \leq b\right) \leq \Pr\left(\sum_{i=1}^n Y_i \leq b\right), \quad (20)$$

where b is an arbitrary integer. Moreover, let A be the first occurrence of success in sequence X_i . Then,

$$\mathbb{E}[A] \leq 1/\eta. \quad (21)$$

Lemma 8 (Lemma 7 in [Chen *et al.*, 2021]). Suppose there is a sequence of $n + 1$ Bernoulli trials: $X_1, X_2, \dots, X_{n+1}$, where the success probability of X_i depends on the preceding trials $X_1, X_2, \dots, X_{i-1}$, and it decreases from 1 to 0. Let $t = \max\{i : \Pr[X_i = 1] \geq \eta\}$, where $0 < \eta < 1$ is a constant. Then, it holds that

$$\Pr[X_i = 1 | X_1 = x_1, X_2 = x_2, \dots, X_{i-1} = x_{i-1}, i \leq t] \geq \eta, \quad (22)$$

where $x_1, \dots, x_{i-1}$ are arbitrary.

Then if $Y_1, \dots, Y_{n+1}$ are independent Bernoulli trials, each with probability η of success, then

$$\Pr\left(\sum_{i=1}^t X_i \leq bt\right) \leq \Pr\left(\sum_{i=1}^t Y_i \leq bt\right), \quad (23)$$

where b is an arbitrary integer.

Lemma 9 (Wald's Equation [Wald, 1945]). Let $(X_n)_{n \in \mathbb{N}}$ be an infinite sequence of real-valued random variables and let N be a nonnegative integer-valued random variable. Assume that: 1) $(X_n)_{n \in \mathbb{N}}$ are integrable (finite-mean) random variables, 2) $\mathbb{E}[X_n 1_{N \geq n}] = \mathbb{E}[X_n]P(N \geq n)$ for every natural number n , 3) the infinite series satisfies $\sum_{n=1}^{\infty} \mathbb{E}[|X_n| 1_{\{N \geq n\}}] < \infty$. Then the random sums $S_N = \sum_{n=1}^N X_n$ and $T_N = \sum_{n=1}^N \mathbb{E}[X_n]$ are integrable and $\mathbb{E}[S_N] = \mathbb{E}[T_N]$.

Lemma 10 (Lemma 11 in [Chen *et al.*, 2021]). Let (Y_i) be a sequence of independent and identically distributed Bernoulli trials, where the success probability is $\beta\epsilon$. Then for a constant integer α , we have

$$\Pr\left[\sum_{i=1}^{\alpha} Y_i > \epsilon\alpha\right] \leq \min\{\beta, e^{-\frac{(1-\beta)^2}{1+\beta}\epsilon\alpha}\}. \quad (24)$$

C Algorithms missed in Section 3

In this section, we recap some subroutines in our algorithms and their performance bounds.

C.1 ThreshSeq Algorithm

We briefly recap the details of the ThreshSeq algorithm (Algorithm 2 in [Chen and Kuhnle, 2024]).

C.2 Unconstrained Submodular Maximization Algorithms

In the LinAdapt algorithm, we adapt the [Fahrbach *et al.*, 2019a]'s algorithm (USM1) that provides a factor of $1/4 - \epsilon$ with constant probability for the USM problem. The guarantees for the USM1 algorithm are stated in Lemma 11.

Lemma 11. For any nonnegative submodular function f and subset $A \subseteq V$, Algorithm 6 outputs a set $S \subseteq V$ in one adaptive round using $O(\log(1/\delta)/\epsilon)$ oracle queries such that with probability at least $1 - \delta$ we have $f(S) \geq (1/4 - \epsilon)\text{opt}_A$, where $\text{opt}_A = \max_{T \subseteq A} f(T)$.

We further adapt an essentially optimal algorithm of [Chen *et al.*, 2019] for USM in our LBA which allows us to improve the approximation ratio slightly.

Theorem 8 ([Chen *et al.*, 2019]). There is an algorithm that achieves a $1/(2 + \epsilon)$ -approximation for USM using $O(\log(1/\epsilon)/\epsilon)$ adaptive rounds and $O(n \log^3(1/\epsilon)/\epsilon^4)$ query complexity.

Algorithm 5: ThreshSeq(V, ϵ, δ)

```
1: Input:  $f, V, k, \epsilon, \delta, \tau$ 
2:  $A \leftarrow \emptyset, A' \leftarrow \emptyset, U \leftarrow V, \Delta = \lceil 4(\frac{2}{\epsilon} \log(n) + \log(\frac{n}{\delta})) \rceil$ 
3: for  $j \leftarrow 1$  to  $\Delta$  do
4:   Update  $U \leftarrow \{x \in U : f(x|A) \geq \tau\}$ 
5:   if  $|U| = 0$  then
6:     return  $A, A'$ 
7:   end if
8:    $U \leftarrow \text{random-permutation}(U)$ 
9:    $s \leftarrow \min\{k - |A|, |V|\}$ 
10:   $B[1 : s] \leftarrow [none, none, \dots, none]$ 
11:  for  $i \leftarrow 1$  to  $s$  (in parallel) do
12:     $T_{i-1} \leftarrow \{v_1, v_2, \dots, v_{i-1}\}$ 
13:    if  $f(v_i|A \cup T_{i-1}) \geq \tau$  then
14:       $B[i] \leftarrow \text{true}$ 
15:    else
16:      if  $f(v_i|A \cup T_{i-1}) < 0$  then
17:         $B[i] \leftarrow \text{false}$ 
18:      end if
19:    end if
20:     $i^* \leftarrow \max\{i : \#\text{trues in } B[1 : i] \geq (1 - \epsilon)i\}$ 
21:     $A \leftarrow A \cup T_{i^*}$ 
22:     $A' \leftarrow A' \cup T_{i^*}$  [where  $B \neq \text{false}$ ]
23:    if  $|A| = k$  then
24:      Return  $A, A'$ 
25:    end if
26:  end for
27: end for
28: Return failure
```

Algorithm 6: USM1(V, ϵ, δ)

```
1 hpt
1: Input:  $f$ , a subset  $A \subseteq V, \epsilon, \delta$ 
2: Set iteration bound  $t \leftarrow \lceil \log(1/\delta) / \log(1 + (4/3)\epsilon) \rceil$ 
3: for  $i = 1$  to  $t$  in parallel do
4:   Let  $R_i$  be a uniformly random subset of  $A$ 
5: end for
6: Set  $S \leftarrow \arg \max_{X \in \{R_1, \dots, R_t\}} f(X)$ 
7: return  $S$ 
```

D Proofs missed of Section 4

D.1 Proofs missed of LinBoundSet (Algorithm 2)

Lemma 1. If LinBoundSet ends successfully, then $f(S) \leq \left(1 + \frac{1+\epsilon}{(1-\epsilon)(1-2\epsilon)\alpha}\right) f(S')$.

Proof of Lemma 1. If $S \leq k$, $S' = S$, thus the Lemma holds. We consider the remaining case $S > k$. Let $\bar{S} = S \setminus S'$. Beside the notations of T_i, T'_{λ_i} in Line 9 of Algorithm 2, we use the following useful notations:

- S_j and T_{j,λ_j^*} are S and T_{λ^*} after iteration j .
- $T_{j,i}$ is the set of first i elements added into T_{j,λ_j^*} .
- $T'_{j,\lambda_i} = T_{j,\lambda_i} \setminus T_{j,\lambda_{i-1}}$.
- $T''_{\lambda_i} = \{v_j \in T'_{\lambda_i} : b[j] \neq \text{false}\}$, i.e., elements in T'_{λ_i} with non-negative marginal gain.
- $T^j = \bigcup_{\lambda_i \leq \lambda_j^*} T''_{\lambda_i}; T''_{\lambda_i} = \bigcup_{\lambda \leq \lambda_i} T''_{\lambda}$.
- $c = \max\{c \in \mathbb{N} : S' \subseteq \bigcup_{j=c}^{\ell} T^j\}$.

By the selection rule in Line 22, Algorithm 2, T''_{λ_i} only contains elements with non-negative marginal gain. If T'_{λ_i} is a **good** block, it has at least $(1-\epsilon)|T'_{\lambda_i}|$ good elements and thus $|T''_{\lambda_i}| \geq (1-\epsilon)|T'_{\lambda_i}|$.

It's easy to see that $|T^j| < k$ for any $j > c$, and the size of T^c may be larger than k . Therefore, we consider two following cases:

Case 1. If $j > c$, $|T^j| \leq k$. By the selection rule of λ^* in Line 21, Algorithm 2, there are $\lambda_j^* - 1$ first blocks in T_{j,λ_j^*} are good and the last one is bad. For any block $T'_{j,\lambda_i} \subseteq T_{j,\lambda_j^*}$, $\lambda_i < \lambda_j^*$, we have

$$\begin{aligned}
 f(T''_{\lambda_i} | S_{j-1} \cup T_{\lambda_{i-1}}^j) &\geq f(T''_{\lambda_i} | S_{j-1} \cup T_{j,\lambda_{i-1}}) && \text{(By the submodularity and } T_{\lambda_{i-1}}^j \subseteq T_{j,\lambda_{i-1}}) \\
 &\geq \sum_{v_l \in T''_{\lambda_i}} f(v_l | S_{j-1} \cup T_{j,l-1}) \\
 &\geq (1-\epsilon)\alpha |T'_{j,\lambda_i}| \frac{M_{\lambda_{i-1}}}{k} && \text{(Since } T'_{j,\lambda_i} \text{ is a good block)} \\
 &\geq (1-\epsilon)\alpha |T'_{j,\lambda_i}| \frac{f(S_{j-1})}{k} && \text{(By selection rule of } M_{\lambda_{i-1}}). \quad (25)
 \end{aligned}$$

Since the block size exponentially increases with factor of $1+\epsilon$ when $\lambda_i \leq k$, so we have $|T'_{j,\lambda_j^*}| \leq \epsilon |T_{j,\lambda_j^*}| / (1+\epsilon)$. Combining this with (25), we have:

$$\begin{aligned}
 f(T^j | S_{j-1}) &= \sum_{\lambda_i \leq \lambda_j^*} f(T''_{\lambda_i} | S_{j-1} \cup T_{\lambda_{i-1}}^j) \\
 &\geq \sum_{\lambda_i < \lambda_j^*} f(T''_{\lambda_i} | S_{j-1} \cup T_{\lambda_{i-1}}^j) && \text{(Since } b[e] \geq 0, \forall e \in T''_{\lambda_i}) \\
 &\geq \sum_{\lambda_i < \lambda_j^*} (1-\epsilon)\alpha |T'_{j,\lambda_i}| \frac{f(S_{j-1})}{k} \\
 &= (1-\epsilon)\alpha |T_{j,\lambda_j^*} \setminus T'_{j,\lambda_j^*}| \frac{f(S_{j-1})}{k} \\
 &\geq \frac{1-\epsilon}{1+\epsilon} \alpha \frac{|T_{j,\lambda_j^*}|}{k} f(S_{j-1}) \\
 &\geq \frac{1-\epsilon}{1+\epsilon} \alpha \frac{|T^j|}{k} f(S_{j-1}).
 \end{aligned}$$

Case 2. If $j = c$, also by the selection rule of λ^* , the last block T'_{c,λ_c^*} is always bad. If $|T_{c,\lambda_c^*}| \leq k$, all previous blocks T'_{c,λ_i} , $\lambda_i < \lambda_c^*$ are good. If $|T_{c,\lambda_c^*}| > k$, some previous contiguous blocks are good and they contain at least k elements. By the selection rule of S , we can let $\lambda_r = \min\{\lambda_r \in \Lambda : \bigcup_{\lambda_r \leq \lambda_i \leq \lambda_c^*} T''_{c,\lambda_i} \subseteq S'\}$. With a note that each block T''_{c,λ_i} contains at most

ϵk elements and S' is the union of some blocks, we have:

$$\begin{aligned}
f(T^c|\bar{S}) &= \sum_{\lambda_r \leq \lambda_i \leq \lambda_c^*} f(T''_{\lambda_i}|\bar{S} \cup T_{\lambda_{i-1}}^c) \\
&\geq \sum_{\lambda_r \leq \lambda_i < \lambda_c^*} f(T''_{\lambda_i}|\bar{S} \cup T_{c, \lambda_{i-1}}) \\
&\geq \sum_{\lambda_r \leq \lambda_i < \lambda_c^*} (1 - \epsilon)\alpha|T'_{c, \lambda_i}| \frac{M_{\lambda_{i-1}}}{k} \\
&\geq \sum_{\lambda_r \leq \lambda_i < \lambda_c^*} (1 - \epsilon)\alpha|T'_{c, \lambda_i}| \frac{f(\bar{S})}{k} \\
&\geq (1 - \epsilon)\alpha|(T_{c, \lambda_j^*} \cap S') \setminus T'_{c, \lambda_j^*}| \frac{f(\bar{S})}{k} \\
&\geq (1 - \epsilon)\alpha \cdot \max\{0, |T^c \cap S'| - \epsilon k\} \frac{f(\bar{S})}{k}.
\end{aligned}$$

Combining two cases above with a note that $k \geq |S'| \geq (1 - \epsilon)k$, we have:

$$\begin{aligned}
f(S) - f(\bar{S}) &= f(T^c|\bar{S}) + \sum_{j=c+1}^{\ell} f(T^j|S_{j-1}) \\
&\geq (1 - \epsilon)\alpha \cdot \max\{0, |T^c \cap S'| - \epsilon k\} \frac{f(\bar{S})}{k} + \sum_{j=c+1}^{\ell} \frac{1 - \epsilon}{1 + \epsilon} \alpha \frac{|T^j|}{k} f(S_{j-1}). \\
&\geq (1 - \epsilon)\alpha \cdot \max\{0, |T^c \cap S'| - \epsilon k\} \frac{f(\bar{S})}{k} + \frac{1 - \epsilon}{1 + \epsilon} \alpha \frac{|S' \setminus T^c|}{k} f(\bar{S}). \\
&\geq \frac{1 - \epsilon}{1 + \epsilon} \alpha (|S'| - \epsilon k) \frac{f(\bar{S})}{k} \\
&\geq \frac{1 - \epsilon}{1 + \epsilon} \alpha (k - 2\epsilon k) \frac{f(\bar{S})}{k} \\
&\geq \frac{1 - \epsilon}{1 + \epsilon} (1 - 2\epsilon) \alpha f(\bar{S})
\end{aligned}$$

which implies that $f(S) \geq (1 + \frac{(1-\epsilon)(1-2\epsilon)\alpha}{1+\epsilon})f(\bar{S})$. By the submodularity of f , we have

$$f(S) \leq f(S') + f(\bar{S}) \implies f(S') \geq f(S) - f(\bar{S}) \geq \frac{(1 - \epsilon)(1 - 2\epsilon)\alpha}{1 + \epsilon + (1 - \epsilon)(1 - 2\epsilon)\alpha} f(S) \quad (26)$$

which implies the proof. $\square$

Lemma 2. $\Pr[\text{iteration } j \text{ fails}] \leq 1/2$.

Proof of Lemma 2. We first show that the following propositions are true.

Proposition 9. For each iteration in `LinBoundSet`, let $\lambda_{(t)} = \max\{\lambda_i \in \Lambda : \lambda_i < t\}$ and $Q_t = \{e \in V : f(e|S \cup T_t) < \alpha M_{\lambda_{(t)}}/k\}$, we have $|Q_0| = 0$, $|Q_{|V|}| = |V|$, and $|Q_t| \leq |Q_{t+1}|$.

By Line 4 of Algorithm 2, we have $f(e|S) \geq \alpha f(S)/k$ for all $e \in V$. Therefore $|Q_0| = 0$. On the other hand, since $f(e|S \cup V) = 0 < \alpha M_{\lambda_{(|V|)}}/k$ for all $e \in V$, $|Q_{|V|}| = |V|$. For the last property, by the submodularity of f and the definition of M_{λ_i} , for any element $e \in Q_t$ we have

$$f(e|S \cup T_{t+1}) \leq f(e|S \cup T_t) < \alpha \frac{M_{\lambda_{(t)}}}{k} \leq \alpha \frac{M_{\lambda_{(t+1)}}}{k}, \quad (27)$$

which implies that $e \in Q_{t+1}$. Hence $Q_t \subseteq Q_{t+1}$ and $|Q_t| \leq |Q_{t+1}|$.

Now, we provide a bound probability that $B[\lambda]$ is false in the following Proposition.

Proposition 10. Let $t = \min\{j \in \mathbb{N} : |Q_j| \geq \beta\epsilon|V|\}$, $\lambda_{(t)} = \max\{\lambda \in \Lambda : \lambda < t\}$ and (Y_j) be the sequence of the independent and identically distributed Bernoulli trials with the success probability is $\beta\epsilon$. For any $\lambda < \lambda_{(t)}$, we have $\Pr[B[\lambda] = \text{false}] \leq \Pr[\sum_{j=1}^{|T'_\lambda|} Y_j > \epsilon|T'_\lambda|]$.

We define an element v_i that is **bad** if $f(v_i|S \cup T_{i-1}) < \alpha M_{\lambda_{(i)}}/k$ and **good** otherwise. Consider the random permutation of V as a sequence of independent Bernoulli trials, with success if the element is bad and failure otherwise.

We have $\Pr[v_i \text{ is bad} | v_1, \dots, v_{i-1}] = |Q_{i-1}|/|V|$ thus the probability that $v_j, j < t$ is bad is less than $\beta\epsilon$. By the definition of $B[\lambda]$ in Line 17, Algorithm 2, the block T'_λ is bad, it must contain more than $\epsilon|T'_\lambda|$ elements. Let $X_i = 1$ if v_i is bad; and $X_i = 0$, otherwise, we have

$$\begin{aligned} \Pr(B[\lambda] = \text{false}) &\leq \Pr(\text{the number of bad elements in } T'_\lambda > \epsilon|T'_\lambda|) \\ &= \Pr\left(\sum_{v_j \in T'_\lambda} X_j > \epsilon|T'_\lambda|\right) \\ &\leq \Pr\left(\sum_{j=1}^{|T'_\lambda|} Y_j > \epsilon|T'_\lambda|\right) \quad (\text{Due to applying Lemma 8}). \end{aligned}$$

By the operation of algorithm 2, the **set S did not change in each iteration of the main loop (Lines 3-24)**. By using Propositions 9, 10 we now prove probability bound in Lemma 2.

The probability iteration j failure is upper bounded by the probability $\lambda^* < t$. We now consider two cases. If $\lambda^* \leq k$, there is at least one block T'_λ with $\lambda \in \Lambda, \lambda < \lambda_{(t)}$ is bad. If $\lambda^* > k$, let $\lambda'_{(t)} = \max\{\lambda' \in \Lambda : \sum_{\lambda \in \Lambda, \lambda' \leq \lambda \leq \lambda_{(t)}} |T'_\lambda| \geq k\}$. Then there must be at least one integer $\lambda \in \Lambda$ between $\lambda_{(t)}$ and $\lambda'_{(t)}$ such that block T'_λ is bad. Therefore, let $B_1 = \{\lambda \in \Lambda : \lambda < k\}$, $B_2 = \{\lambda \in \Lambda : |\Lambda \cap [\lambda, \lambda_{(t)}]| \leq \lceil 1/\epsilon \rceil\}$, then we have

$$\Pr(\text{iteration } j \text{ fails}) \leq \Pr[\lambda^* < t] \leq \Pr(\exists \lambda \in B_1 \cup B_2, B[\lambda] = \text{false}) \leq \frac{1}{2}. \quad (28)$$

The proof of inequality (28) is based on some properties of Bernoulli trials, which is shown in detail in [Chen *et al.*, 2021]. For the sake of completeness, we write down the details of the proof below.

By Lemma 9 we have

$$\Pr[\text{iteration } j \text{ fails}] \leq \Pr[\exists \lambda \in B_1 \cup B_2 \text{ with } B[\lambda] = \text{false}] \quad (29)$$

$$= \mathbb{E}[1_{\{\exists \lambda \in B_1 \cup B_2 \text{ with } B[\lambda] = \text{false}\}}] \quad (30)$$

$$\leq \mathbb{E}\left[\sum_{\lambda \in B_1} 1_{\{B[\lambda] = \text{false}\}} + \sum_{\lambda \in B_2} 1_{\{B[\lambda] = \text{false}\}}\right] \quad (31)$$

$$\leq \mathbb{E}\left[\sum_{\lambda \in B_1} 1_{\{B[\lambda] = \text{false}\}}\right] + \mathbb{E}\left[\sum_{\lambda \in B_2} 1_{\{B[\lambda] = \text{false}\}}\right] \quad (32)$$

$$= \mathbb{E}\left[\sum_{\lambda \in B_1} \mathbb{E}[1_{\{B[\lambda] = \text{false}\}}]\right] + \mathbb{E}\left[\sum_{\lambda \in B_2} \mathbb{E}[1_{\{B[\lambda] = \text{false}\}}]\right] \quad (33)$$

where (33) holds, since the sequence $(1_{\{B[\lambda] = \text{false}\}})$ and the random variable t follows the assumptions in Lemma 9: 1) $1_{\{B[\lambda] = \text{false}\}}$ s are all integrable random variables because they only take the values 0 and 1; 2) t is a stopping time since it only depends on the previous $t - 1$ selections; 3) $\Pr[1_{\{t \geq n\}} = 0] = 1$ for any $n \geq |V|$.

The first term of (33) is bounded by adapting Proposition 10 as follows:

$$\mathbb{E} \left[\sum_{\lambda \in B_1} \mathbb{E}[1_{\{B[\lambda]=\text{false}\}}] \right] \leq \mathbb{E} \left[\sum_{\lambda \in B_1} \leq \Pr \left(\sum_{j=1}^{|T'_\lambda|} Y_j > \epsilon |T'_\lambda| \right) \right] \quad (34)$$

$$\leq \mathbb{E} \left[\sum_{\lambda \in \{\lfloor (1+\epsilon)^u \rfloor : u \geq 1\}} \Pr \left(\sum_{j=1}^{|T'_\lambda|} Y_j > \epsilon |T'_\lambda| \right) \right] \quad (35)$$

$$\leq \sum_{\lambda \in \{\lfloor (1+\epsilon)^u \rfloor : u \geq 1\}} \min\{\beta, e^{-\frac{(1-\epsilon)^2}{1+\beta} \epsilon |T'_\lambda|}\} \quad (\text{By Lemma 10}) \quad (36)$$

$$\leq \sum_{\lambda \geq 1} \min\{\beta, e^{-\frac{\epsilon \lambda}{2}}\} \quad (\text{Due to } \beta < 1/2, |T'_\lambda| < \lambda) \quad (37)$$

$$\leq \alpha\beta + \sum_{\lambda=a+1}^{\infty} e^{-\frac{\epsilon \lambda}{2}}, \quad (\text{where } a = \lfloor \frac{1}{8\beta} \rfloor = \lfloor \frac{2}{\epsilon} \log(\frac{8}{1-e^{-\epsilon/2}}) \rfloor) \quad (38)$$

$$\leq \alpha\beta + \frac{e^{-\epsilon(a+1)/2}}{1-e^{-\epsilon/2}} \quad (39)$$

$$\leq \frac{1}{8} + \frac{1}{8} = \frac{1}{4}, \quad (40)$$

where (35) follows from $B_1 \subseteq \{\lfloor (1+\epsilon)^u \rfloor : u \geq 1\}$, and let $\{\lfloor (1+\epsilon)^u \rfloor : u \geq 1\} = \{\lambda_1, \lambda_2, \dots\}$, $|T'| = \lambda_i - \lambda_{i-1}$;

For the second term in (33), from Proposition 10, Lemma 10 and the fact that $|B_2| < \lceil 1/\epsilon \rceil$, we have

$$\mathbb{E} \left[\sum_{\lambda \in B_2} \mathbb{E}[1_{\{B[\lambda]=\text{false}\}}] \right] \leq \mathbb{E} \left[\sum_{\lambda \in B_2} \leq \Pr \left(\sum_{j=1}^{|T'_\lambda|} Y_j > \epsilon |T'_\lambda| \right) \right] \leq \beta \lceil 1/\epsilon \rceil \leq \frac{1}{4}$$

Put it into (33), we have

$$\Pr[\text{iteration } j \text{ fails}] < \frac{1}{4} + \frac{1}{4} = \frac{1}{2}$$

□

Theorem 3. For any $\epsilon, \delta \in (0, 1)$, the algorithm LinBoundSet runs in $O(\log(n/\delta)/\epsilon^2)$, returns two sets S and S' such that the following holds unconditionally: (1) The algorithm succeeds with a probability of at least $1 - \delta/n$; (2) The algorithm takes $O(n/\epsilon^3)$ query complexity in expectation; (3) If the algorithm succeeds, it returns S, S' satisfying: for any subset $X \subseteq V$, $|X| \leq k$, we have $f(S \cup X) \leq \left(2 + \alpha + \frac{1}{\alpha} + (1 + \frac{1}{\alpha}) \frac{2(2-\epsilon)}{(1-\epsilon)(1-2\epsilon)} \epsilon\right) f(S')$.

Proof of Theorem 3. Prove the success probability. Since each successful iteration will remove $(\beta\epsilon)$ -fraction of V thus if the LinBoundSet fails, there are no more than $\lceil \log_{1/(1-\beta\epsilon)}(n) \rceil$ successful iterations. Let X be the number of successes in the ℓ iterations. Then, By Lemma 2, X can be regarded as a sum of dependent Bernoulli trials, where the success probability is larger than $1/2$. Let Y be a sum of independent Bernoulli trials, where the success probability is equal to $1/2$. By applying Lemma 10, we have

$$\begin{aligned} \Pr[\text{LinBoundSet fails}] &\leq \Pr[X \leq \lceil \log_{1/(1-\beta\epsilon)}(n) \rceil] \\ &\leq \Pr[Y \leq \lceil \log_{1/(1-\beta\epsilon)}(n) \rceil] \quad (\text{By Lemma 10}) \\ &\leq \Pr \left[Y \leq \frac{\log(n)}{\beta\epsilon} \right] \\ &\leq e^{-\frac{1}{2} \left(\frac{2\beta\epsilon+1}{2(\beta\epsilon+1)} \right)^2 2(1+\frac{1}{\beta\epsilon}) \log(\frac{n}{\delta})} \quad (\text{By Lemma 6}) \\ &\leq \left(\frac{\delta}{n} \right)^{\frac{(2\beta\epsilon+1)^2}{4\beta\epsilon(\beta\epsilon+1)}} \\ &\leq \frac{\delta}{n}. \end{aligned}$$

Prove the bound of $f(X \cup S)$. We use the notation in the proof of Lemma 1. We now further prove that $f(X \cup S) \leq (\alpha + 1)f(S)$. For each element $x \in X \setminus S$, define $j(x) + 1$ be the iteration where x is filtered out (Line 4, Algorithm 2), we have $f(x|S_{j(x)}) \leq \alpha f(S_{j(x)})/k$. Therefore,

$$\begin{aligned} f(X \cup S) - f(S) &\leq \sum_{x \in X \setminus S} f(x|S_{j(x)}) \\ &\leq \sum_{x \in X \setminus S} \alpha \frac{f(S_{j(x)})}{k} \\ &\leq \alpha f(S_{j(x)}) \quad (\text{Since } |X| \leq k) \\ &\leq \alpha f(S). \end{aligned}$$

Combine this with (26), we obtain

$$\begin{aligned} f(X \cup S) &\leq (\alpha + 1)f(S) \leq (\alpha + 1) \left(1 + \frac{1 + \epsilon}{(1 - \epsilon)(1 - 2\epsilon)\alpha} \right) f(S') \\ &= \left(2 + \alpha + \frac{1}{\alpha} + (1 + \frac{1}{\alpha}) \frac{2(2 - \epsilon)}{(1 - \epsilon)(1 - 2\epsilon)} \epsilon \right) f(S') \end{aligned}$$

Prove the adaptive and query complexities. LinBoundSet requires the oracle queries on Lines 4, 10, 12 of Algorithm 2. At these times, the queries are executed in parallel, there are constant adaptive rounds in each iteration or the main loop. Thus, the algorithm needs $O(\ell) = O(\log(n/\delta)/\epsilon^3)$ adaptive complexity.

Let V_j be the set V after Line 4 at iteration j and let Y_i be the number of iterations between $(i - 1)$ -th success and i -th success. By Lemma 7, we have $\mathbb{E}[Y_i] \leq 2$. At each iteration j , the algorithm takes $|V_{j-1}| + 1$ queries on Line 4, takes $|\Lambda| \leq |V_j|$ queries on Line 10 and takes total $|V_j|$ queries after the second and the third loops. Therefore, the number of queries is bounded by

$$\begin{aligned} \sum_{j=1}^{\ell} \mathbb{E}(|V_{j-1}| + 1 + 2|V_j|) &\leq n + \ell + \sum_{j=1}^{\ell} 3|V_j| \\ &\leq n + \ell + 3 \sum_{i=1}^{\infty} [Y_i(1 - \epsilon\beta)^i n] \\ &\leq n + \ell + 3 \frac{n}{\beta\epsilon} \\ &= O\left(\frac{n}{\epsilon^3}\right). \end{aligned}$$

This completes the proof. $\square$

E Algorithms and Proofs missed of Section 5

LinAst constructs $O(\frac{1}{\epsilon} \log(\frac{1}{\epsilon}))$ solutions in the main loop with one adaptive round (Lines 4-9, Algorithm 7). For each iteration, the algorithm calls ThreshSeq twice sequentially to get candidate solutions A'_i, B'_i with a threshold τ_i related to the guess of optimal value.

Then, LinAst finds another solution by using USM2 algorithm of [Chen *et al.*, 2019] for USM over the ground set A'_i . Finally, it returns the best among the obtained solutions (Line 10, Algorithm 7).

E.1 LinAst (Algorithm 7) and proof of Theorem 5

Algorithm 7: LinAst

- 1: **Input:** f, V, k, ϵ .
 - 2: $S_0 \leftarrow \text{LinAdapt}(f, V, k, 1, \epsilon, 1/3)$, $a \leftarrow 12 + (\frac{16}{1-4\epsilon} + \frac{8(2-\epsilon)}{(1-\epsilon)(1-2\epsilon)})\epsilon$
 - 3: $c \leftarrow 6 + \epsilon$, $\ell \leftarrow \lceil \log_{1-\epsilon}(\frac{1}{a}) \rceil + 1$, $\delta \leftarrow 1/3$, $M \leftarrow af(S_0)/(ck)$
 - 4: **for** $i \leftarrow 1$ **to** ℓ (in parallel) **do**
 - 5: $\tau_i \leftarrow M(1 - \epsilon)^i$
 - 6: $A_i, A'_i \leftarrow \text{ThreshSeq}(f(\cdot), V, k, \tau_i, \epsilon, \delta)$
 - 7: $B_i, B'_i \leftarrow \text{ThreshSeq}(f(\cdot), V \setminus A_i, k, \tau_i, \epsilon, \delta)$
 - 8: $A''_i \leftarrow \text{USM2}(A'_i, \epsilon')$
 - 9: $C_i \leftarrow \arg \max\{f(A'_i), f(B'_i), f(A''_i)\}$
 - 10: **end for**
 - 11: $S \leftarrow \arg \max_{X \in \{C_i\}_{i=1}^{\ell} \cup \{S_0\}} f(X)$
 - 12: **Return** S
-

Theorem 5. For any input (f, V, k, ϵ) , where $\epsilon \in (0, 1)$, LinAst runs in $O(\log(n)/\epsilon^2)$ adaptive complexity and $O(n/\epsilon^3)$ query complexity and returns an approximation ratio of $1/6 - \epsilon$ in expectation with probability of at least $1 - 1/n$.

Proof of Theorem 5. LinAst calls LinAdapt to find S_0 . This task takes $O(\frac{\log n}{\epsilon^2})$ adaptivity and $O(\frac{n}{\epsilon^3})$ queries. Then, the algorithm runs in $2\ell = 2(\lceil \log_{1-\epsilon}(\frac{1}{a}) \rceil + 1) = O(\frac{1}{\epsilon} \log(\frac{1}{\epsilon}))$ iterations in one adaptive round. Each iteration takes $O(\frac{n}{\epsilon})$ query complexity and $O(\frac{1}{\epsilon} \log(\frac{n}{\delta}))$ adaptive rounds to call ThreshSeq algorithm. Therefore, the adaptive complexity of LinAst is at most

$$O(\frac{\log n}{\epsilon^2}) + O(\frac{1}{\epsilon} \log(\frac{n}{\delta})) = O(\frac{\log(n)}{\epsilon^2})$$

and its query complexity is

$$O(\frac{n}{\epsilon^3}) + O(\frac{1}{\epsilon} \log(\frac{1}{\epsilon}) \frac{n}{\epsilon}) = O(\frac{n}{\epsilon^3}).$$

The probability that LinAdapt fails is at most $1/(3n)$. In the main loop of LinAst, each iteration returns sets (A_i, A'_i) (or (A_i, A'_i)) with fail probability at most $1/(3n)$. Therefore, the algorithm returns the set C as the final solution with a successful probability of at least $1 - 1/n$ by the union bound. By Theorem 2, we have $f(S_0) \leq \text{opt} \leq af(S_0)$, where $a = (12 + (\frac{16}{1-4\epsilon} + \frac{8(2-\epsilon)}{(1-\epsilon)(1-2\epsilon)})\epsilon)$. Thus $\frac{f(S_0)}{ck} \leq \frac{\text{opt}}{ck} \leq \frac{af(S_0)}{ck} = M$. Besides, $M(1-\epsilon)^i \in [\frac{f(S_0)}{ck}, \frac{af(S_0)}{ck}]$. Therefore, there exist an integer i so that

$$\frac{(1-\epsilon)\text{opt}}{ck} \leq M(1-\epsilon)^i \leq \frac{\text{opt}}{ck}.$$

If the algorithm succeeds, the proof of approximation ratio is similar to the proof of Theorem 7 in [Chen and Kuhnle, 2024]. For the sake of completeness, we write down the details of the proof by following. Denote τ_i is the value of τ at iteration i in LinAst.

Case 1: If $|A| = k$ or $|B| = k$. By the theoretical guarantee of ThreshSeq (Theorem 1), we have

$$\begin{aligned} f(C) &\geq \max\{f(A'), f(B')\} \geq (1-\epsilon)\tau_i \\ &\geq \frac{(1-\epsilon)^2}{c} \text{opt} \geq (\frac{1}{6} - \epsilon) \text{opt}. \end{aligned}$$

Case 2: If $|A| < k$ and $|B| < k$. By Theorem 1, for any element $e \in V$ we have $f(e|A) \leq \tau_i$ and $f(e|B) \leq \tau_i$. By the submodularity of f , we have

$$f(O \cup A) - f(A) \leq \sum_{e \in O} f(e|A) < k\tau_i \leq \frac{\text{opt}}{c}. \quad (41)$$

$$f((O \setminus A) \cup B) - f(B) \leq \sum_{e \in O \setminus A} f(e|B) < k\tau_i \leq \frac{\text{opt}}{c}. \quad (42)$$

Combine two inequalities (41)-(42) with the fact that $A \cap B = \emptyset$, we have

$$f(A') + f(B') \geq f(A) + f(B) \quad (43)$$

$$\geq f(O \cup A) + f((O \setminus A) \cup B) - \frac{2\text{opt}}{c} \quad (44)$$

$$\geq f(O \setminus A) + f(O \cup A \cup B) - \frac{2\text{opt}}{c} \quad (\text{Due to the submodularity}) \quad (45)$$

$$\geq f(O \setminus A) - \frac{2\text{opt}}{c} \quad (\text{Due to the non-negativity}) \quad (46)$$

Besides, the USM2 gives approximation ratio of $1/(2+\epsilon)$, we have $\mathbb{E}[f(A'')] \geq f(O \cup A)$. Combine this with (46), we have

$$\begin{aligned} \text{opt} &\leq f(O) \leq f(O \cup A) + f(O \setminus A) \\ &\leq (2+\epsilon)\mathbb{E}[f(C)] + 2f(C) + \frac{2\text{opt}}{c}. \end{aligned}$$

It follows that

$$\mathbb{E}[f(C)] \geq \frac{\text{opt}}{c} \geq (\frac{1}{6} - \epsilon) \text{opt}$$

which completes the proof. $\square$

E.2 LinAtg Algorithm (Algorithm 8) and proof of Theorem 4

LinAtg works in a different way from LinAst. It sequentially constructs two pairs of disjoint sets (A, B) and (A', B') in two main loops which contain at most $\ell = O(\frac{1}{\epsilon} \log(\frac{1}{\epsilon}))$ iterations (Lines 5-14, Algorithm 8). At each iteration, the algorithm calls ThreshSeq over appropriate ground sets to select two batches of elements with high marginal gain (S, S') and adds them into (A, A') (or (B, B')), respectively. It then selects a candidate solution by adapting the USM2 algorithm of [Chen *et al.*, 2019] over the ground set A' . Finally, LinAtg returns the best one among the obtained feasible solutions (Line 16, Algorithm 8).

Algorithm 8: LinAtg

```

1: Input:  $f, V, k, \epsilon$ .
2:  $S_0 \leftarrow \text{LinAdapt}(f, V, k, 1, \epsilon, 1/3)$ ,  $a \leftarrow 12 + (\frac{16}{1-4\epsilon} + \frac{8(2-\epsilon)}{(1-\epsilon)(1-2\epsilon)})\epsilon$ 
3:  $c \leftarrow 8/\epsilon$ ,  $\epsilon' \leftarrow (1 - 1/e)\epsilon/8$ ,  $\ell \leftarrow \lceil \log_{1-\epsilon'}(\frac{1}{ac}) \rceil + 1$ ,  $\delta \leftarrow 1/(3\ell)$ ,  $M \leftarrow af(S_0)/k$ 
4:  $A \leftarrow \emptyset$ ,  $A' \leftarrow \emptyset$ ,  $B \leftarrow \emptyset$ ,  $B' \leftarrow \emptyset$ 
5: for  $i \leftarrow 1$  to  $\ell$  do
6:    $\tau \leftarrow M(1 - \epsilon')^{i-1}$ 
7:    $A, S' \leftarrow \text{ThreshSeq}(f(A \cup \cdot), A, k - |A|, V, \epsilon', \delta, \tau)$ 
8:    $A \leftarrow A \cup S$ ,  $A' \leftarrow A' \cup S'$ 
9:   If  $|A| = k$  then break
10: end for
11: for  $i \leftarrow 1$  to  $\ell$  do
12:    $\tau \leftarrow M(1 - \epsilon')^{i-1}$ 
13:    $S, S' \leftarrow \text{ThreshSeq}(f(B \cup \cdot), V \setminus A, k - |B|, \epsilon', \delta, \tau)$ 
14:    $B \leftarrow B \cup S$ ,  $B' \leftarrow B' \cup S'$ 
15:   If  $|B| = k$  then break
16: end for
17:  $A'' \leftarrow \text{USM2}(A', \epsilon')$ 
18:  $C \leftarrow \arg \max\{f(A'), f(B'), f(A''), f(S_0)\}$ 
19: Return  $C$ 

```

Theorem 4. For any input (f, V, k, ϵ) , where $\epsilon \in (0, 1)$, LinAtg runs in $O(\log(1/\epsilon) \log(n)/\epsilon^2)$ adaptive complexity and $O(n/\epsilon^3)$ query complexity, returns an approximation ratio of $0.193 - \epsilon$ in expectation with probability of at least $1 - 1/n$.

Proof of Theorem 4. Prove the complexities. The algorithm LinAdapt takes $O(\frac{\log n}{\epsilon^2})$ adaptivity and $O(\frac{n}{\epsilon^3})$ queries to call LinAdapt. Then, the algorithm runs two loops, each loop takes $\ell = \log_{\frac{1}{1-\epsilon'}}(ac) + 1 = O(\frac{1}{\epsilon} \log(\frac{1}{\epsilon}))$ sequential iterations. In each iteration, it takes $O(\frac{n}{\epsilon})$ query complexity and $O(\frac{1}{\epsilon} \log(\frac{n}{\delta}))$ to call ThreshSeq algorithm, where $\delta = 1/(3\ell)$. Therefore, the adaptive complexity LBA is

$$\begin{aligned}
O\left(\frac{\log n}{\epsilon^2}\right) + O\left(\frac{1}{\epsilon} \log\left(\frac{1}{\epsilon}\right) \frac{1}{\epsilon} \log\left(\frac{n}{\delta}\right)\right) &= O\left(\frac{1}{\epsilon^2} \log\left(\frac{1}{\epsilon}\right) \log\left(\frac{n}{\epsilon} \log\left(\frac{1}{\epsilon}\right)\right)\right) \\
&= O\left(\frac{1}{\epsilon^2} \log\left(\frac{1}{\epsilon}\right) (\log(n) + \log\left(\frac{1}{\epsilon} \log\left(\frac{1}{\epsilon}\right)\right))\right) \\
&= O\left(\frac{\log(n)}{\epsilon^2} \log\left(\frac{1}{\epsilon}\right)\right).
\end{aligned}$$

and its query complexity is

$$O\left(\frac{n}{\epsilon^3}\right) + O\left(\frac{1}{\epsilon} \log\left(\frac{1}{\epsilon}\right) \frac{n}{\epsilon}\right) = O\left(\frac{n}{\epsilon^3}\right).$$

The probability that LinAdapt fails is at most $1/n$, and the probability that ThreshSeq fails in each iteration is at most δ . By the union probability, the probability BoostAdapt fails is at most $1/n + 2\ell \cdot \delta = 1/n$.

Prove the approximation ratio. ATG algorithm in [Chen and Kuhnle, 2024] uses $f(e_{max})$ to give the bound $f(O)/k \leq f(e_{max}) \leq f(O)$. Our LinAtg uses $M = af(S_0)/k$ to give a bound of opt/k . By Theorem 2, we have $f(S_0) \leq \text{opt} \leq af(S_0)$, where $a = \left(12 + (\frac{16}{1-4\epsilon} + \frac{8(2-\epsilon)}{(1-\epsilon)(1-2\epsilon)})\epsilon\right)$. Thus

$$\frac{M}{a} = \frac{f(S_0)}{k} \leq \frac{\text{opt}}{k} \leq \frac{af(S_0)}{k} = M$$

implying that $M(1 - \epsilon')^i \in [\frac{M}{ca}, M] \supseteq [\frac{\text{opt}}{ck}, \frac{\text{opt}}{k}]$. From the main loops, LBA works similarly to ATG. Therefore, we follow the proof of Theorem 8 in [Chen and Kuhnle, 2024]. For the sake of completeness, we write down the details of the proof below. The notations used in the proof are listed below.

- A' and A' are returned by the first loop of Algorithm 8, while B and B' are returned by the second one.
- Define A_i as A after iteration i .
- Let $\mathcal{A}'_j$ be the first j elements in A , where $1 \leq j \leq |A|$. Furthermore, for $|A'| < j \leq k$, let $\mathcal{A}'_j$ be A' combined with $k - |A'|$ dummy elements.
- Let $\{a'_j\} = \mathcal{A}'_j \setminus \mathcal{A}'_{j-1}$, a'_j be returned at iteration $i(j)$, and $A_{i(j)}$ be the set A returned at iteration $i(j)$. If $\{a'_j\}$ is dummy element, let $i(j) = \ell + 1$. Then, we likewise define $B_i, \mathcal{B}'_j, B_{i(j)}$.

Lemma 12 (Lemma 9 in [Chen and Kuhnle, 2024]). *For $1 \leq j \leq k$, there are at least $\lceil (1 - \epsilon')k \rceil$ of j such that*

$$f(\mathcal{A}'_j) - f(\mathcal{A}'_{j-1}) + \frac{M}{ck} \geq \frac{1 - \epsilon}{k} (f(O \cup A_{i(j)-1}) - f(\mathcal{A}'_{j-1})).$$

And for any j ,

$$f(\mathcal{A}'_j) \geq f(\mathcal{A}'_{j-1}).$$

Lemma 13 (Lemma 10 in [Chen and Kuhnle, 2024]). *For $1 \leq j \leq k$, there are at least $\lceil (1 - \epsilon')k \rceil$ of j such that*

$$f(\mathcal{B}'_j) - f(\mathcal{B}'_{j-1}) + \frac{M}{ck} \geq \frac{1 - \epsilon}{k} (f((O \setminus A) \cup B_{i(j)-1}) - f(\mathcal{B}'_{j-1})).$$

And for any j ,

$$f(\mathcal{B}'_j) \geq f(\mathcal{B}'_{j-1}).$$

Lemma 14 (Lemma 11 in [Chen and Kuhnle, 2024]). *Let $\Gamma_u = f(\mathcal{A}'_{j(u)}) + f(\mathcal{B}'_{j(u)})$, where $j(u)$ is the u -th j which satisfies Lemma 12 or Lemma 13. Then, there are at least $\lceil (1 - \epsilon')k \rceil$ of u follow that*

$$f(O \setminus A) - \Gamma_u - \frac{2M}{c(1 - \epsilon')} \leq \left(1 - \frac{1 - \epsilon'}{k}\right) \left(f(O \setminus A) - \Gamma_{u-1} - \frac{2M}{c(1 - \epsilon')}\right).$$

Lemma 14 yields a recurrence of the form $(b - u_{i+1}) \leq a(b - u_i)$, $u_0 = 0$, and has a solution $u_i \geq b(1 - a^i)$. Consequently, we have

$$\begin{aligned} f(A') + f(B') &\geq \left(1 - \left(1 - \frac{1 - \epsilon'}{k}\right)^{(1 - \epsilon')k}\right) \left(f(O \setminus A) - \frac{2M}{c(1 - \epsilon')}\right) \\ &\geq \left(1 - e^{-(1 - \epsilon')^2}\right) \left(f(O \setminus A) - \frac{2M}{c(1 - \epsilon')}\right). \end{aligned}$$

Let $\beta = 1 - e^{-(1 - \epsilon')^2}$. From the choice of C on line 16, Algorithm 8, we have $2f(C) \geq f(A') + f(B')$ and from (47), we have

$$\begin{aligned} f(O \setminus A) &\leq \frac{2}{\beta} f(C) + \frac{2M}{c(1 - \epsilon')^2} \\ &\leq \frac{2}{\beta} f(C) + \frac{2f(O)}{c(1 - \epsilon')^2}. \end{aligned}$$

Assume USM2 has a ratio of $1/\rho$ for USM problem. For any A , $f(O \cap A)/\rho \leq \mathbb{E}[f(A'')|A]$; therefore

$$f(O \cap A) \leq \rho \mathbb{E}[f(C)].$$

For any set A , $f(O) \leq f(O \cap A) + f(O \setminus A)$ by the submodularity and nonnegativity. Thus

$$\begin{aligned} f(O) &\leq f(O \cap A) + f(O \setminus A) \\ &\leq \frac{2}{\beta} f(C) + \frac{2f(O)}{c(1 - \epsilon')^2} + \rho. \end{aligned}$$

Therefore

$$f(C) \geq \frac{1 - \frac{2}{c(1 - \epsilon')^2}}{\rho + \frac{2}{\beta}} f(O) \geq \left(\frac{e - 1}{\rho(e - 1) + 2e} - \epsilon\right) f(O).$$

By replacing $\rho = 2 + \epsilon$ for USM2, we obtain the ratio. □

F Proofs missed of Section 6

We recap the following notations used for analyzing the BoostAdapt's performance guarantees

- Supposing X and Y ordered: $X = \{x_1, x_2, \dots, x_{|X|}\}$, $Y = \{y_1, y_2, \dots, y_{|Y|}\}$, we conduct: $X^i = \{x_1, x_2, \dots, x_i\}$, $Y^i = \{y_1, y_2, \dots, y_i\}$, where x_i (or y_i) is i -th element added into X (or Y).
- For $e \in X \cup Y$, we assume that e is added into X or Y at iteration i_e .
- X_i and Y_i are X and Y after the iteration i of the main loop and $X_0 = Y_0 = \emptyset$. Then, we define X'_i and Y'_i analogously.
- τ_X^i and τ_Y^i are τ_X and τ_Y after the iteration i of the main loop of BoostAdapt.
- τ_X^{last} and τ_Y^{last} are τ_X and τ_Y at the last iterations of the main loop of BoostAdapt when X and Y are considered to update.
- Define A_i^+ (res. B_i^+) as the set of elements in A_i (res. B_i) having the marginal gain at least τ_X (res. τ_Y) at iteration i , $X_i^+ = \cup_{j=1}^i A_j^+$, $Y_i^+ = \cup_{j=1}^i B_j^+$ and $X^+ = X_\Delta^+$, $Y^+ = Y_\Delta^+$.
- For an element $e \in X \cup Y$, we denote by $X^{<e}$, $Y^{<e}$ as X, Y right before e is selected into X or Y , respectively.

Lemma 3. a) For any iteration i , we have $f(X'_i) \geq f(X_i)$ and $f(Y'_i) \geq f(Y_i)$.

b) At the end of BoostAdapt, we have $f(X'') \geq (1 - \epsilon)f(X')$ and $f(Y'') \geq (1 - \epsilon)f(Y')$.

Proof of 3. a) We prove the Lemma by the induction hypothesis. For $i = 1$, we have $f(X'_1) = f(A'_1) \geq f(A_1) = f(X_1)$ due to the Theorem 1, so the Proposition holds. Assuming the proposition holds for j , we have:

$$f(X'_{j+1}) = f(A'_{j+1} \cup X'_j) = f(A'_{j+1}|X'_j) + f(X'_j) \quad (47)$$

$$\geq f(A'_{j+1}|X_j) + f(X_j) \quad (48)$$

$$\geq f(A_{j+1}|X_j) + f(X_j) = f(X_{j+1}) \quad (49)$$

where the inequality (48) is due to the submodularity of f and $X'_j \subseteq X_j$, inequality (49) is due to the theoretical bound of ThreshSeq (Theorem 1). Therefore, by the induction hypothesis, we have $f(X'_i) \geq f(X_i)$ for any iteration i .

By similar reasoning, we can prove $f(Y'_i) \geq f(Y_i)$ for any iteration i , which completes the proof of **a)**.

b). If $|X'| = k$, then $X'' = X'$ the Lemma holds. If $|X'| > k$, by the setting of k' and the selection rule of X'' , we get $|X''| = k \geq k'(1 - \epsilon) \geq |X|(1 - \epsilon) \geq |X'|(1 - \epsilon)$. Therefore $|X' \setminus X''| \leq \epsilon|X'|$. Elements in $|X' \setminus X''|$ have lowest marginal so we obtain

$$\begin{aligned} f(X') &= \sum_{e \in X'} f(e|X'^{<e}) = \sum_{e \in X''} f(e|X'^{<e}) + \sum_{e \in X' \setminus X''} f(e|X'^{<e}) \\ &\leq \sum_{e \in X''} f(e|X'^{<e}) + \epsilon \sum_{e \in X'} f(e|X'^{<e}) \\ &\leq \sum_{e \in X''} f(e|X''^{<e}) + \epsilon f(X') \quad (\text{Due to the submodularity and } X'' \subseteq X') \\ &= f(X'') + \epsilon f(X') \end{aligned}$$

which implies $f(X'') \geq (1 - \epsilon)f(X')$. By similar reasoning, we can prove $f(Y'') \geq (1 - \epsilon)f(Y')$. This completes the proof of **b)**. $\square$

Lemma 4. If BoostAdapt succeeds and $X_1 = \emptyset$, after iteration i the following properties hold:

(a) If i is odd and $|X_i| < k'$: for any sets $C \subseteq Y_{i-1}^+$ and $C' \subseteq Y_{i-1} \setminus Y_{i-1}^+$, we have

$$\sum_{x \in C} f(x|X_i) \leq \sum_{x \in C} \frac{f(e|Y^{<e})}{1 - \epsilon} \quad \text{and} \quad \sum_{e \in C'} f(e|X_i) = \frac{\epsilon}{1 - \epsilon} f(Y_{i-1}^+).$$

(b) If i is even and $|Y_i| < k'$, for any set $D \subseteq X_{i-1}^+$ and $D' \subseteq X_{i-1} \setminus X_{i-1}^+$, we have

$$\sum_{x \in D} f(x|Y_i) \leq \sum_{x \in D} \frac{f(e|X^{<e})}{1 - \epsilon} \quad \text{and} \quad \sum_{e \in D'} f(e|Y_i) = \frac{\epsilon}{1 - \epsilon} f(X_{i-1}^+).$$

Proof of Lemma 4. a) For $i \geq 1$, i is odd. If $i = 1$, $X_1 = Y_0 = \emptyset$. Thus $C = \emptyset$ and the Lemma holds. If $i \geq 3$, let $C = C_2 \cup C_4 \cup \dots \cup C_{i-1}$, where C_j ($j \leq i-1$) is the set of elements in C added into Y_j^+ , i.e., $C_j = B_j^+ \cap C$. Each element

$e \in C_j$ is selected into Y_j^+ at iteration $j \leq i-1$ and has not been added yet into X at the previous iteration. By the Theorem 1 and the submodularity of f , for any $e \in C_j$ we have

$$f(e|X_i) \leq f(e|X_{j-1}) < \tau_X^{j-1} = \frac{\tau_Y^j}{1-\epsilon} \leq \frac{f(e|Y^{<e})}{1-\epsilon}.$$

It follows that

$$\begin{aligned} \sum_{e \in C} f(e|X_i) &= \sum_{j=2,4,\dots,i-1} \sum_{e \in C_j} f(e|X_i) \\ &< \sum_{j=2,4,\dots,i-1} \sum_{e \in C_j} \frac{f(e|Y^{<e})}{1-\epsilon} \\ &= \sum_{e \in C} \frac{f(e|Y^{<e})}{1-\epsilon}. \end{aligned}$$

Each element $e \in B_j \setminus B_j^+$ also has not been added yet into X at the previous iteration, so we have

$$f(e|X_i) \leq f(e|X_{j-1}) < \tau_X^{j-1} = \frac{\tau_Y^j}{1-\epsilon}.$$

Since $|B_j^+| \geq (1-\epsilon)|B_j|$, $|B_j \setminus B_j^+| \leq \epsilon|B_j| \leq \frac{\epsilon}{1-\epsilon}|B_j^+|$. Besides, due to the fact that $Y_i = \bigcup_{j=1}^i B_j$ and $Y_i^+ = \bigcup_{j=1}^i B_j^+$, therefore we have:

$$\sum_{e \in C'} f(e|X_i) = \sum_{j=2,4,\dots,i-1} \sum_{e \in C' \cap (B_j \setminus B_j^+)} f(e|X_i) \quad (50)$$

$$< \sum_{j=2,4,\dots,i-1} \sum_{e \in B_j \setminus B_j^+} \frac{\tau_Y^j}{1-\epsilon} \quad (51)$$

$$\leq \sum_{j=2,4,\dots,i-1} \epsilon|B_j| \frac{\tau_Y^j}{1-\epsilon} \quad (52)$$

$$\leq \sum_{j=2,4,\dots,i-1} \frac{\epsilon}{1-\epsilon}|B_j^+| \frac{\tau_Y^j}{1-\epsilon} \quad (53)$$

$$\leq \frac{\epsilon}{1-\epsilon} \sum_{j=2,4,\dots,i-1} \sum_{e \in B_j^+} \frac{f(e|Y^{<e})}{1-\epsilon} \quad (54)$$

$$= \frac{\epsilon}{1-\epsilon} f(Y_{i-1}^+). \quad (55)$$

b) We prove this case by the same reasoning with part **a)**

If $i \geq 2$, i is even. If $i = 2$, since $X_1 = \emptyset$, thus $D = \emptyset$ and the Lemma holds. If $i \geq 4$, let $D = D_1 \cup D_3 \cup \dots \cup D_{i-1}$, where $D_j = A_j^+ \cap D$. Each $e \in D$ is selected into X at iteration j , $3 \leq j \leq i-1$ and has not been added yet into Y at the previous iteration. By the Theorem 1, for any $e \in D_j$ we have

$$f(e|Y_i) \leq f(e|Y_{j-1}) < \tau_Y^{j-1} = \frac{\tau_X^j}{1-\epsilon} \leq \frac{f(e|X^{<e})}{1-\epsilon} \quad (56)$$

Therefore

$$\begin{aligned} \sum_{e \in D} f(e|Y_i) &= \sum_{j=3,5,\dots,i-1} \sum_{e \in D_j} f(e|Y_i) \\ &< \sum_{j=3,5,\dots,i-1} \sum_{e \in D_j} \frac{f(e|X^{<e})}{1-\epsilon} \\ &\leq \sum_{e \in D} \frac{f(e|X^{<e})}{1-\epsilon}. \end{aligned}$$

Each element $e \in A_j \setminus A_j^+$ also has not been added yet into Y at the previous iteration, so we have $f(e|X_i) \leq f(e|X_{j-1}) < \tau_X^{j-1} = \frac{\tau_Y^j}{1-\epsilon}$. We also we have:

$$\begin{aligned}
\sum_{e \in D'} f(e|X_i) &= \sum_{j=2,4,\dots,i-1} \sum_{e \in D' \cap (A_j \setminus A_j^+)} f(e|Y_i) \\
&< \sum_{j=2,4,\dots,i-1} \sum_{e \in A_j \setminus A_j^+} \frac{\tau_X^j}{1-\epsilon} \\
&\leq \sum_{j=2,4,\dots,i-1} \epsilon |A_j| \frac{\tau_X^j}{1-\epsilon} \\
&\leq \sum_{j=2,4,\dots,i-1} \frac{\epsilon}{1-\epsilon} |A_j^+| \frac{\tau_X^j}{1-\epsilon} \\
&\leq \frac{\epsilon}{1-\epsilon} \sum_{j=2,4,\dots,i-1} \sum_{e \in X_j^+} \frac{f(e|X^{<e})}{1-\epsilon} \\
&= \frac{\epsilon}{1-\epsilon} f(X_{i-1}^+)
\end{aligned}$$

which completes the proof. $\square$

Using Lemma 4 and the fact that if $|T| = k$, then $|T \setminus O| \geq |O \setminus T|$, where $T \in \{X^+, Y^+\}$, we can bound of the optimal solution in some cases related to the size of X and Y in Lemma 5.

Lemma 5 If BoostAdapt succeeds and $X_1 = \emptyset$, at the end BoostAdapt we have

- a) If $|X| < k'$ and $|Y| < k'$, then $f(O) < \frac{4f(S)}{(1-\epsilon)^2(1-2\epsilon)}$.
- b) If there exists $T \in \{X, Y\}$ such that $|T| = k'$, then $f(S) \geq (1-\epsilon)^4 \frac{\text{opt}}{4}$.

Proof of Lemma 5. We consider following cases:

Case 1. If $|X| < k'$ and $|Y| < k'$, by the definition of Δ in Algorithm 3, we have $\frac{M}{4k} \geq \tau(1-\epsilon)^i \geq \frac{\epsilon(1-\epsilon)f(S_0)}{k}$. Thus $\tau_X^{\text{last}} \leq \epsilon f(S_0)/k \leq \epsilon \text{opt}/k$. Note that

By the submodularity and applying Lemma 4 and Theorem 1, we have

$$f(X \cup O) - f(X) \leq \sum_{e \in O \setminus X} f(e|X) \quad (57)$$

$$= \sum_{e \in Y^+ \cap O} f(e|X) + \sum_{e \in (Y \setminus Y^+) \cap O} f(e|X) + \sum_{e \in O \setminus (X \cup Y^+)} f(e|X) \quad (58)$$

$$< \frac{\sum_{e \in Y^+ \cap O} f(e|Y^{<e})}{1-\epsilon} + \frac{\epsilon}{1-\epsilon} f(Y^+) + k\tau_X^{\text{last}} \quad (59)$$

$$\leq \frac{f(Y^+)}{1-\epsilon} + \frac{\epsilon}{1-\epsilon} f(Y^+) + k\tau_X^{\text{last}} \quad (60)$$

$$\leq \frac{1+\epsilon}{1-\epsilon} f(Y') + \epsilon \text{opt} \quad (61)$$

where inequality (57) is due to the submodularity of f ; inequality (59) is due to: applying Lemma 4 with $C = Y^+ \cap O$, and $C' = (Y \setminus Y^+) \cap O$ and applying Theorem 1; inequality (61) due to Lemma 3, i.e., $f(Y') \geq f(Y^+)$.

Similarity, since $X_1 = \emptyset$ and by applying Lemma 4 and Theorem 1 we have

$$f(Y \cup O) - f(Y) \leq \sum_{e \in O \setminus Y} f(e|Y) \quad (\text{Due to the submodularity of } f) \quad (62)$$

$$= \sum_{e \in O \cap X^+} f(e|Y) + \sum_{e \in (X \setminus X^+) \cap O} f(e|Y) + \sum_{e \in O \setminus (X^+ \cup Y)} f(e|Y) \quad (63)$$

$$< \frac{\sum_{e \in O \cap X^+} f(e|X^{<e})}{1 - \epsilon} + \frac{\epsilon}{1 - \epsilon} f(X^+) + k\tau_Y^{last} \quad (64)$$

$$\leq \frac{f(X^+)}{1 - \epsilon} + \frac{\epsilon}{1 - \epsilon} f(X^+) + k\tau_Y^{last} \quad (65)$$

$$\leq \frac{1 + \epsilon}{1 - \epsilon} f(X') + \epsilon \text{opt}. \quad (66)$$

Combining two inequalities (61) and (66) and the selection rule of the final solution we have

$$\begin{aligned} f(O) &\leq f(O \cup X) + f(O \cup Y) && (\text{Due to the submodularity of } f) \\ &< \frac{1 + \epsilon}{1 - \epsilon} (f(X') + f(Y')) + f(X) + f(Y) + 2\epsilon \text{opt} \\ &= \frac{2}{1 - \epsilon} (f(X') + f(Y')) + 2\epsilon \text{opt} \\ &\leq \frac{2}{(1 - \epsilon)^2} (f(X'') + f(Y'')) + 2\epsilon \text{opt} && (\text{Due to Lemma 7}) \\ &\leq \frac{4f(S)}{(1 - \epsilon)^2} + 2\epsilon \text{opt} \end{aligned}$$

which implies the proof of a).

Case 2. If $|X| = k'$ and $|Y| = k'$. Since $X_1 = \emptyset$ we must have $i_{x_{k'}} > 1$. If $i_{y_{k'}} = 2$, we have

$$f(S) \geq f(Y_2') \geq (1 - \epsilon)|Y_2| \tau_Y^2 \geq \frac{(1 - \epsilon)^3 \text{opt}}{4}.$$

thus the Lemma holds.

If $i_{y_{k'}} > 2$, then each element e that is not added into X at iteration $i_{x_{k'}}$, also has not been added into X at iteration $i_{x_{k'}} - 2$.

By applying Lemma 4 with a note that $|X_{i_{x_{k'}}-2}| < k'$, we have $f(e|X) < \tau_X^{i_{x_{k'}}-2}$. Therefore

$$f(X \cup O) - f(X) \leq \sum_{e \in O \setminus X} f(e|X) \quad (67)$$

$$= \sum_{e \in Y_{i_{x_{k'}}-1}^+ \cap O} f(e|X) + \sum_{e \in (Y_{i_{x_{k'}}-1} \setminus Y_{i_{x_{k'}}-1}^+) \cap O} f(e|X) + \sum_{e \in O \setminus (X \cup Y_{i_{x_{k'}}-1}^+)} f(e|X^{<e}) \quad (68)$$

$$< \frac{\sum_{e \in Y_{i_{x_{k'}}-1}^+ \cap O} f(e|Y^{<e})}{1 - \epsilon} + \frac{\epsilon}{1 - \epsilon} f(Y_{i_{x_{k'}}-1}^+) + |O \setminus X| \cdot \tau_X^{i_{x_{k'}}-2}. \quad (69)$$

$$\leq \frac{\sum_{e \in Y \cap O} f(e|Y^{<e})}{1 - \epsilon} + \frac{\epsilon}{1 - \epsilon} f(Y^+) + |O \setminus X| \cdot \tau_X^{i_{x_{k'}}-2}. \quad (70)$$

where inequality (69) is due to: applying Lemma 4 with $C = Y_{i_{x_{k'}}-1}^+ \cap O$, and $C' = (Y_{i_{x_{k'}}-1} \setminus Y_{i_{x_{k'}}-1}^+) \cap O$ and applying Theorem 1.

Similarly, each element e is not added to Y at iteration $i_{y_{k'}}$, also is not added to Y at iteration $i_{y_{k'}} - 2$. By Lemma 4, we have:

$$f(Y \cup O) - f(Y) \leq \sum_{e \in O \cap X^+} f(e|Y) + \frac{\epsilon}{1 - \epsilon} f(X^+) + \sum_{e \in O \setminus (Y \cup X_{i_{y_{k'}}-1}^+)} f(e|Y) \quad (71)$$

$$\leq \frac{\sum_{e \in O \cap X^+} f(e|X^{<e})}{1 - \epsilon} + \frac{\epsilon}{1 - \epsilon} f(X^+) + |O \setminus Y| \cdot \tau_Y^{i_{y_{k'}}-2}. \quad (72)$$

On the other hand, since $|X| = k' > k \geq |O|$. Since at each iteration i , ThreshSeq selects at least $(1 - \epsilon)|A_i|$ element with marginal gain is above the threshold, so $|A_i^+| \geq (1 - \epsilon)|A_i|$. Thus $|X^+| \geq (1 - \epsilon)|X| = (1 - \epsilon)k'$. If $(1 - \epsilon)k'$ is an integer, then $(1 - \epsilon)k' = k$ and thus $|X^+| = k$. If $(1 - \epsilon)k'$ is not an integer, then $(1 - \epsilon)k' < k$, $\lceil (1 - \epsilon)k' \rceil = k$. Since $|X^+|$ is an integer, we have $|X^+| = \lceil (1 - \epsilon)k' \rceil = k$. Overall, $|X^+| = k$ which implies $|X^+ \setminus O| \geq |O \setminus X^+| \geq |O \setminus X'| \geq |O \setminus X|$. By the definition of X', X^+ we have

$$\begin{aligned} \sum_{e \in X^+ \setminus O} f(e|X^{<e}) &\geq |X^+ \setminus O| \cdot \tau_X^{i_{x_{k'}}} \\ &\geq (1 - \epsilon)^2 |O \setminus X^+| \tau_X^{i_{x_{k'}} - 2} \\ &\geq (1 - \epsilon)^2 |O \setminus X| \tau_X^{i_{x_{k'}} - 2} \\ \Rightarrow |O \setminus X| \tau_X^{i_{x_{k'}} - 2} &\leq \frac{\sum_{e \in X^+ \setminus O} f(e|X^{<e})}{(1 - \epsilon)^2}. \end{aligned}$$

Similarly, since $|Y| = k', |Y^+| \geq k$ and $|Y^+ \setminus O| \geq |O \setminus Y^+|$ we have

$$\sum_{e \in Y^+ \setminus O} f(e|Y^{<e}) \geq (1 - \epsilon)^2 |O \setminus Y| \tau_Y^{i_{y_k} - 2} \quad (73)$$

$$\Rightarrow |O \setminus Y| \tau_Y^{i_{y_k} - 2} \leq \frac{\sum_{e \in Y^+ \setminus O} f(e|Y^{<e})}{(1 - \epsilon)^2}. \quad (74)$$

Combining (70), (72), (73), (74), the fact that $f(X^+) \leq f(X'), f(Y^+) \leq f(Y')$ and Lemma 3 we get

$$\begin{aligned} f(O) &\leq f(O \cup X) + f(O \cup Y) \\ &\leq \frac{\sum_{e \in Y^+ \cap O} f(e|Y^{<e})}{1 - \epsilon} + \frac{\sum_{e \in Y^+ \setminus O} f(e|Y^{<e})}{(1 - \epsilon)^2} \\ &\quad + \frac{\sum_{e \in O \cap X^+} f(e|X^{<e})}{1 - \epsilon} + \frac{\sum_{e \in X^+ \setminus O} f(e|X^{<e})}{(1 - \epsilon)^2} + \frac{\epsilon}{1 - \epsilon} (f(X^+) + f(Y^+)) + f(X) + f(Y) \\ &\leq \frac{\sum_{e \in Y^+} f(e|Y^{<e})}{(1 - \epsilon)^2} + \frac{\sum_{e \in X^+} f(e|X^{<e})}{(1 - \epsilon)^2} + \frac{f(X') + f(Y')}{1 - \epsilon} \\ &\leq \frac{\sum_{e \in Y'} f(e|Y^{<e})}{(1 - \epsilon)^2} + \frac{\sum_{e \in X'} f(e|X^{<e})}{(1 - \epsilon)^2} + \frac{f(X') + f(Y')}{1 - \epsilon} \\ &\leq \frac{f(Y')}{(1 - \epsilon)^2} + \frac{f(X')}{(1 - \epsilon)^2} + \frac{f(X') + f(Y')}{1 - \epsilon} \\ &< \frac{2f(Y')}{(1 - \epsilon)^2} + \frac{2f(X')}{(1 - \epsilon)^2} = 2 \frac{f(X') + f(Y')}{(1 - \epsilon)^2} \\ &\leq 2 \frac{f(X'') + f(Y'')}{(1 - \epsilon)^3} \quad (\text{Due to Lemma 3}). \end{aligned}$$

Case 3. If $|Y| = k'$ and $|X| < k'$. If $i_{y_k} = 2$, we have

$$f(S) \geq f(Y_2) \geq (1 - \epsilon)^2 \tau_Y^2 \geq (1 - \epsilon)^2 \frac{\text{opt}}{4}$$

the Lemma holds. If $i_{y_k} > 2$, similar to the transformation of **Case 2** of this proof, we also have

$$f(Y \cup O) - f(Y) \leq \frac{\sum_{e \in O \cap X^+} f(e|X^{<e})}{1 - \epsilon} + \frac{\epsilon}{1 - \epsilon} f(X^+) + |O \setminus Y| \cdot \tau_Y^{i_{y_{k'}} - 2} \quad (75)$$

$$\leq \frac{\sum_{e \in O \cap X^+} f(e|X^{<e})}{1 - \epsilon} + \frac{\epsilon}{1 - \epsilon} f(X^+) + \frac{\sum_{e \in Y^+ \setminus O} f(e|Y^{<e})}{(1 - \epsilon)^2} \quad (76)$$

where inequality (76) is due to the fact that

$$|O \setminus Y| \tau_Y^{i_{y_k} - 2} \leq \frac{\sum_{e \in Y^+ \setminus O} f(e|Y^{<e})}{(1 - \epsilon)^2}. \quad (77)$$

Since $|X| < k'$, similar to the transformations of **Case 1**, we have

$$\begin{aligned}
f(X \cup O) - f(X) &\leq \sum_{e \in O \setminus X} f(e|X) \\
&= \sum_{e \in Y^+ \cap O} f(e|X) + \sum_{e \in (Y \setminus Y^+) \cap O} f(e|X) + \sum_{e \in O \setminus (X \cup Y^+)} f(e|X) \\
&< \frac{\sum_{e \in Y^+ \cap O} f(e|Y^{<e})}{1 - \epsilon} + \frac{\epsilon}{1 - \epsilon} f(Y^+) + k\tau_X^{last}. \\
&\leq \frac{\sum_{e \in Y^+ \cap O} f(e|Y^{<e})}{1 - \epsilon} + \frac{\epsilon}{1 - \epsilon} f(Y') + \epsilon \text{opt}.
\end{aligned}$$

It follows that

$$\begin{aligned}
f(O) &\leq f(O \cup Y) + f(O \cup X) \\
&< \frac{\sum_{e \in Y^+ \cap O} f(e|Y^{<e})}{1 - \epsilon} + \frac{\sum_{e \in Y^+ \setminus O} f(e|Y^{<e})}{(1 - \epsilon)^2} + \frac{\sum_{e \in O \cap X^+} f(e|X^{<e})}{1 - \epsilon} \\
&\quad + \frac{\epsilon}{1 - \epsilon} (f(X') + f(Y')) + f(X) + f(Y) + \epsilon \text{opt} \\
&\leq \frac{\sum_{e \in Y^+} f(e|Y^{<e})}{(1 - \epsilon)^2} + \frac{\sum_{e \in X^+} f(e|X^{<e})}{1 - \epsilon} + \frac{\epsilon}{1 - \epsilon} (f(X') + f(Y')) + f(X) + f(Y) + \epsilon \text{opt} \\
&< \frac{f(Y')}{(1 - \epsilon)^2} + \frac{f(X')}{1 - \epsilon} + \frac{\epsilon}{1 - \epsilon} (f(X') + f(Y')) + f(X) + f(Y) + \epsilon \text{opt} \\
&< 2 \frac{f(Y') + f(Y')}{(1 - \epsilon)^2} + \epsilon \text{opt} \\
&\leq 2 \frac{f(X'') + f(Y'')}{(1 - \epsilon)^3} + \epsilon \text{opt}.
\end{aligned}$$

Case 4. If $|Y| < k'$ and $|X| = k'$. Since $X_1 \neq \emptyset$ we must have $i_{x_k} > 1$. Similar to the **Case 2**, we have

$$f(X \cup O) - f(X) < \frac{\sum_{e \in Y^+ \cap O} f(e|Y^{<e})}{1 - \epsilon} + \frac{\epsilon}{1 - \epsilon} f(Y^+) + \frac{\sum_{e \in X^+ \setminus O} f(e|X^{<e})}{(1 - \epsilon)^2}.$$

Since $|Y| < k'$ similar the **Case 1**, we have

$$f(Y \cup O) - f(Y) \leq \frac{\sum_{e \in O \cap X^+} f(e|X^{<e})}{1 - \epsilon} + \frac{\epsilon}{1 - \epsilon} f(X^+) + k\tau_Y^{last}.$$

Similar to the transformation at the end of **Case 3**, we have

$$\begin{aligned}
f(O) &\leq f(O \cup X) + f(O \cup Y) \\
&< \frac{\sum_{e \in Y'} f(e|Y^{<e})}{1 - \epsilon} + \frac{\sum_{e \in X'} f(e|X^{<e})}{(1 - \epsilon)^2} + \frac{\epsilon}{1 - \epsilon} (f(X') + f(Y')) + f(X) + f(Y) + \epsilon \text{opt}. \\
&\leq 2 \frac{f(X'') + f(Y'')}{(1 - \epsilon)^3} + \epsilon \text{opt}.
\end{aligned}$$

Combine **Case 2**, **Case 3**, **Case 4**, we have:

$$f(O) \leq 2 \frac{f(X'') + f(Y'')}{(1 - \epsilon)^4} \leq \frac{4f(S)}{(1 - \epsilon)^4}$$

which completes the proof of b). □

Proof of Theorem 6. Prove successful probability and complexities. BoostAdapt first calls LinAdapt to find S_0 in $O(\frac{\log n}{\epsilon^2})$ adaptivity and $O(\frac{n}{\epsilon^3})$ queries. Then, the algorithm runs in $\Delta = O(\frac{1}{\epsilon} \log(\frac{k}{\epsilon}))$ iterations; each takes $O(\frac{n}{\epsilon})$ query complexity and $O(\frac{1}{\epsilon} \log(\frac{n}{\delta}))$ adaptive rounds to call ThreshSeq algorithm. Therefore, the adaptive complexity BoostAdapt is

$$\begin{aligned}
O(\frac{\log n}{\epsilon^2}) + O\left(\frac{1}{\epsilon} \log(\frac{k}{\epsilon}) \frac{1}{\epsilon} \log(\frac{n}{\delta})\right) &= O\left(\frac{1}{\epsilon^2} \log(\frac{k}{\epsilon}) \log(\frac{n}{\epsilon} \log(\frac{k}{\epsilon}))\right) \\
&= O\left(\frac{1}{\epsilon^2} \log(\frac{k}{\epsilon}) \log(\frac{n}{\epsilon}) + \frac{1}{\epsilon^2} \log(\frac{k}{\epsilon}) \log(\frac{k}{\epsilon})\right) \\
&= O\left(\frac{1}{\epsilon^2} \log(\frac{k}{\epsilon}) \log(\frac{n}{\epsilon})\right)
\end{aligned}$$

and its query complexity is

$$O\left(\frac{n}{\epsilon^3}\right) + O\left(\frac{1}{\epsilon} \log\left(\frac{k}{\epsilon}\right) \frac{n}{\epsilon}\right) = O\left(\frac{n}{\epsilon^2} \log\left(\frac{k}{\epsilon}\right)\right)$$

The probability that LinAdapt fails is at most $1/(3n)$, and the probability that ThreshSeq fails in each iteration is at most δ . By the union probability, the probability BoostAdapt fails is at most $1/(3n) + \Delta \cdot \delta = 1/n$. $\square$

G Additional Experimental Details and Results

In this section, we elaborate on the experimental configuration and provide additional empirical findings along with some discussions.

G.1 Algorithms and Settings

We compare our algorithms LinAst, LinAtg and BoostAdapt with state-of-the-arts for non-monotone SMC including:

- **IteratedGreedy (IG)**: The algorithm in [Gupta *et al.*, 2010] achieves $1/6 - \epsilon$ approximation ratio when the $1/2$ -approximation of [Buchbinder *et al.*, 2012] is used for the unconstrained maximization subproblem. The algorithm takes $O(nk)$ query and adaptive complexities.
- **ParSSP**: The algorithm in [Cui *et al.*, 2023] that runs in $O(\log^2 n)$ adaptivity, $O(nk \log^2 n)$ query complexity and returns a $1/4 - \epsilon$ approximation solution in expectation.
- **AdaptiveSimpleThreshold (AST)**: The algorithm in [Chen and Kuhnle, 2024] achieves $1/6 - \epsilon$ approximation ratio with $O(\log(n/\delta)/\epsilon + \log(1/\epsilon)/\epsilon)$ adaptivity complexity and $O(\log_{1-\epsilon}(1/(6k)) \cdot (n/\epsilon + n \log^3(1/\epsilon)/\epsilon^4))$ query complexity.
- **AdaptiveThresholdGreedy (ATG)**: The algorithm in [Chen and Kuhnle, 2024] achieves $0.193 - \epsilon$ approximation ratio with $O(\log(n) \log(k))$ adaptivity complexity and $O(n \log(k))$ query complexity.
- **FASTLS+GUIDEDRG (FLS)**: The algorithm in [Chen *et al.*, 2024a] achieves $0.385 - \epsilon$ approximation ratio with $O(kn/\epsilon)$ query complexity.
- **FastRandomGreedy (FRG)**: The algorithm in [Buchbinder *et al.*, 2015] achieves $1/e - \epsilon$ approximation ratio with $O(k)$ adaptivity complexity and $O(n)$ query complexity.
- **INTERPOLATEDGREEDY (PITG)**: The algorithm in [Chen *et al.*, 2025] achieves $1/e - \epsilon$ approximation ratio with $O(\ell^3 \epsilon^{-2} \log(n) \log(k))$ adaptivity complexity and $O(\ell^4 \epsilon^{-2} n \log(n) \log(k))$ query complexity.

The comparison is about **four metrics**: object values, adaptive complexity, number of queries, and running time. We experimented with two well-known applications: Revenue Maximization (RM) and Maximum Cut (MC) [Chen and Kuhnle, 2024; Kuhnle, 2019; Amanatidis *et al.*, 2020].

We set $\epsilon = 0.1$ for all algorithms, and other settings are set the same as [Chen and Kuhnle, 2024]. Furthermore, we use an algorithm in [Fahrbach *et al.*, 2019a] (USM1), which returns a ratio of $1/4 - \epsilon$ for our LinAst, LinAtg and algorithms (AST, ATG, ANM) in [Chen and Kuhnle, 2024; Fahrbach *et al.*, 2023] and set the same setting as [Chen and Kuhnle, 2024; Fahrbach *et al.*, 2023]. However, the variations across different α values remain relatively small, indicating low empirical sensitivity. In our experiments, we evaluated several values of $\alpha \in \{1, 2, 3, 4\}$ and selected the one achieving the best empirical performance. We fix $\alpha = 4$ as it provides the best empirical trade-off, while $\alpha = 1$ is theoretically optimal and yields nearly comparable performance in practice. Note that α is treated as a constant parameter, and this empirical setting does not affect any of the theoretical guarantees or analyses established in the paper. For PITG, we set $\ell = 5$, following the experimental configuration in [Chen *et al.*, 2025]. However, with $\ell = 5$, the approximation ratio of the algorithm PITG is no longer guaranteed to be preserved.

G.2 Applications and settings

The applications utilized in the experiments are defined as follows:

Maximum Cut Application. Given graph $G = (V, E)$, and nonnegative edge weight w_{ij} for each edge $(i, j) \in E$. For $S \subseteq V$, let

$$f(S) = \sum_{i \in V \setminus S} \sum_{j \in S} w_{ij}.$$

The objective of the problem is to find a solution S such that $f(S)$ is maximized while ensuring that the cardinality of S is less than k . The function $f(\cdot)$ is a submodular and non-monotone function.

Revenue Maximization Application. Considering a graph $G = (V, E)$ representing a social network, where each edge $(i, j) \in E$ is associated with a non-negative weight w_{ij} , we adopt the concave graph model introduced by [Hartline *et al.*, 2008]. In this model, each user $i \in V$ is linked to a non-negative, a concave function $f_i : \mathbb{R}^+ \rightarrow \mathbb{R}^+$. The function $v_i(S) = f_i\left(\sum_{j \in S} w_{ij}\right)$ indicates the likelihood of user i purchasing a product if the set S adopts it. Thus, the total revenue from seeding a set S is given by

$$f(S) = \sum_{i \in V \setminus S} f_i \left(\sum_{j \in S} w_{ij} \right). \quad (78)$$

The objective of the application is to find a solution S such that $f(S)$ is maximized while ensuring that the cardinality of S is less than k . The function $f(\cdot)$ is a submodular and non-monotone function.

Table 2: Details of datasets used in the experiments.

Dataset Name	Nodes	Edges	Types	Applications	Sources\References
Barabási-Albert	968	5,708	Undirected	MC	[Chen and Kuhnle, 2024]
ca-GrQc	5,242	14,496	Undirected	MC	[Leskovec <i>et al.</i> , 2007]
ca-Astro	18,772	198,110	Undirected	RM	[Leskovec <i>et al.</i> , 2007]
web-Google	875,713	5,105,039	Undirected	MC	[Leskovec <i>et al.</i> , 2009]

Other settings. In our experimental setup, we employed OpenMP to parallelize the code written in C++. To precisely measure the execution time of these algorithms, we utilized **Chrono**, a standard component of C++. Our methodology includes marking the start and end points of each algorithm’s execution using Chrono and computing the time gap between these two points. In the Maximum Cut, we uniformly set the weight of all edges to 1.0. In the Revenue Maximization, we randomly assigned weights to all edges from the interval (0, 1). Besides, we experimented on a high-performance computing (HPC) server cluster with the following parameters: partition=large, #threads(CPU)=64, node=2, max memory = 3,073 GB.

G.3 Additional results

Figure 2 presents the performance of the compared algorithms on the Barabási–Albert dataset for the RM application, while Figure 3 illustrates the results on the Astro dataset for the MC application. Furthermore, Figures 4 and 5 focus on the largest dataset, Google, evaluating algorithm performance across various values of k for the MC application. It is important to note that the results of the ParSSP algorithm are omitted on the Google dataset due to its impracticality when handling large-scale data and large values of k .

Objective value. Figures 2-5(a) show the objective values. They show that our BoostAdapt lines consistently reach the highest points with every k . LinAtg approximates to BoostAdapt while LinAst is a little lower than BoostAdapt. With Barabási-Albert and Astro, except AST and ParSSP, other algorithms are also consistent with ours. AST and FLS are moderately lower than ours. Significantly, ParSSP always hits the lowest values. With Google, the gap between ours with FRG, AST, and ParSSP seems larger, especially in the case of small k . Especially, Figure 4(a) indicates ours are much higher than FRG, AST and ParSSP. In this case, our algorithms can be up to 1.3-1.6 times larger than FRG and AST. Finally, our algorithms significantly improve the solution quality.

Adaptive rounds. In RM (Figure 2(b)), FRG marks the lowest points, which means it saves the best adaptive rounds. Being consistent to FRG they are LinAst and AST. Moderately higher than these lines they are LinAtg, IG and BoostAdapt. Significantly, ParSSP, and ATG are much higher than the others. In the MC application (Figures 3–5(b)), the algorithms ATG, IG, and PITG exhibit the highest numbers of adaptive rounds. Specifically, PITG records the highest value on the Astro dataset, while ATG and IG dominate in the Google dataset under small and large k settings, respectively. Notably, our proposed algorithm LinAst consistently achieves the lowest number of adaptive rounds across all settings, and our other algorithms also maintain values close to this minimum. In addition, both AST and FRG demonstrate significantly lower adaptivity compared to the aforementioned high-adaptivity baselines.

Number of queries. In both the RM and MC applications (Figures 2–3(c)), our proposed algorithms consistently achieve a minimal number of oracle queries. Specifically, in the RM setting (Figure 2(c)), the group of algorithms LinAtg, FRG, ParSSP, and BoostAdapt exhibit the lowest query counts, closely followed by LinAst, ANM, ParSSP, FLS, and IG. In contrast, AST, PITG, and ATG incur significantly higher query costs. In the MC application (Figure 3(c)), FLS yields the highest number of queries, followed by PITG, IG, and FRG, while the remaining algorithms maintain substantially lower query counts. The change also happens with the experiment of Google. With small k , PITG and IG waste much higher than the others. Our algorithms always mark the lowest points. With bigger k , LinAtg still hit the lowest points. LinAst, ATG and AST are close to LinAtg while BoostAdapt grows when k grows. With high values of k , BoostAdapt reaches to FRG. IG wastes the highest number of queries. Finally, LinAtg and LinAst always keep steadily low while BoostAdapt keep low values in most cases except Google with large k .

× AST + ATG □ BoostAdapt ○ FLS — FRG — IG ⊖ LinAST ⊖ LinATG ··· ParSSP — PITG

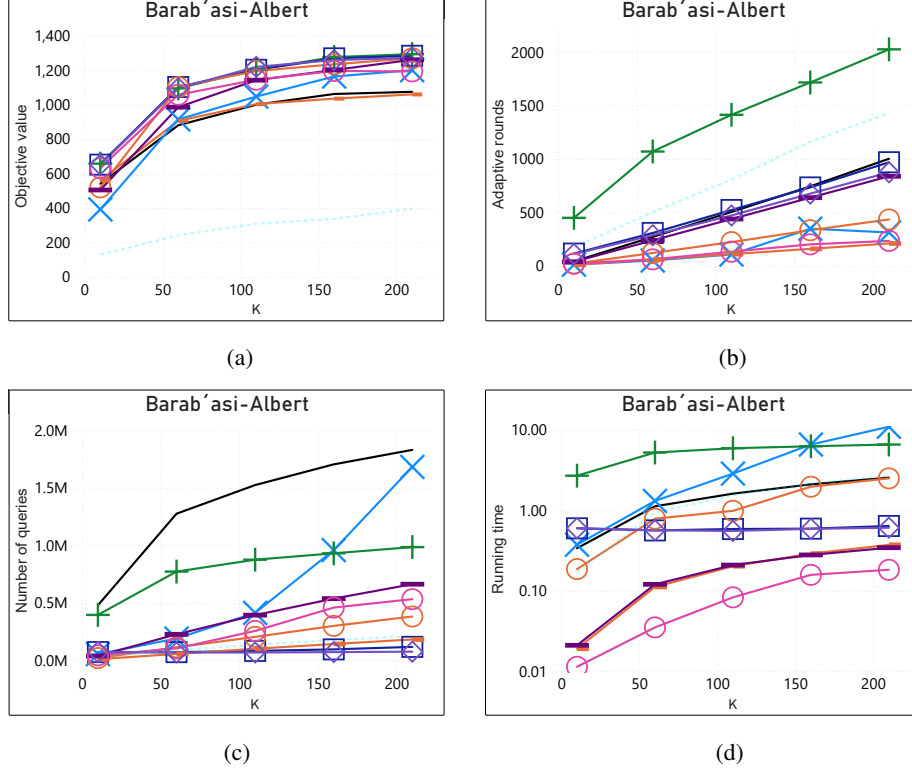


Figure 2: Performance of algorithms for Revenue Maximization.

Time taken. In the RM (Figure 2(d)), LinAst has the lowest execution time, followed by the IG and FRG algorithms. Our algorithms BoostAdapt and LinAtg have the same running time, which is higher than IG and FRG but lower than the remaining. In MC (Figure 3(d)), all our algorithms have the lowest points, a little lower than ATG and IG. FRG, FLS, and PITG are moderately higher than the above algorithms while ParSSP has essentially highest values.

On the Google dataset (Figure 4 - 5), LinAst always runs fastest while others fluctuate. With small k , our BoostAdapt and LinAtg are almost equal to AST, higher than FRG and IG, and lower than ATG. With bigger k , IG runs slower than BoostAdapt and LinAtg and LinAst and have the trend run faster than the remaining.

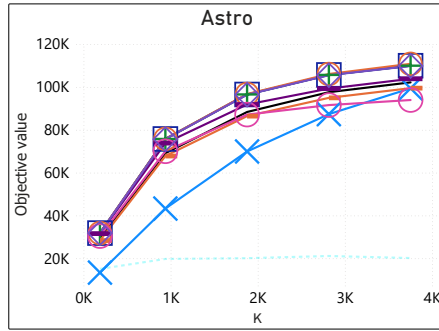
The above metrics show that our algorithms outperform the others when keeping the best quality solutions, wasting the lowest number of queries within acceptable query adaptivity.

H More detail reviews for SMC algorithms

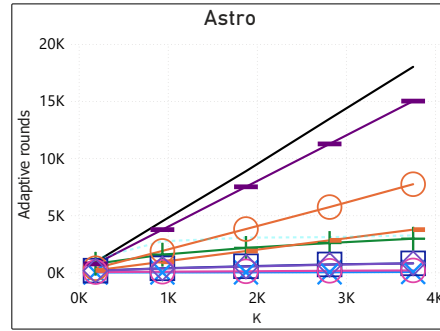
People studied non-adaptive methods for SMC first. [Nemhauser *et al.*, 1978] showed the best approximation algorithm with a factor of $1 - 1/e$ based on the sequential greedy approach. However, the sequential searching of greedy made time running too slow if the input increased. It led other works to try to reduce the time consumption problem. In [Badanidiyuru and Vondrák, 2014], the authors proposed a deterministic approximation algorithm to reduce time consumption down to $O(n \log n)$ by streaming fashion model. Significantly, [Kuhnle, 2021b] and [Li *et al.*, 2022] have simultaneously proposed a linear-time approximation algorithm with a tight factor of nearly $1 - 1/e$ recently. However, these algorithms cannot work for non-monotone functions.

To work with the approach of parallel, the first major contribution belonged to [Balkanski and Singer, 2018] when first applied the adaptive sampling method for the monotone SMC problem with constant 3-approximation ratio and $O(\log n)$ adaptivity. Their algorithm filtered a fixed fraction of the elements out of the ground set to get a quadratic query complexity. In contrast, [Breuer *et al.*, 2020] sped up the sampling method to enhance the approximation factor to $1 - 1/e$ within $O(\log n \log k)$ adaptive rounds and $O(n \log(\log k))$ queries. For the non-monotone objective function, the first popular approach to solving the SMC problem in practice is to design approximation algorithms with low query complexity. The greedy algorithm was an effective approach for submodular optimization problems. It sequentially selects elements with the largest marginal gain and explores the diminishing return property to get performance bounds. [Gupta *et al.*, 2010] first developed an iterated greedy algorithm with $1/6$ ratio in $O(nk)$ queries for SMC. The work of [Lee *et al.*, 2010] latter improve the ratio to $1/4 - \epsilon$ by developing a

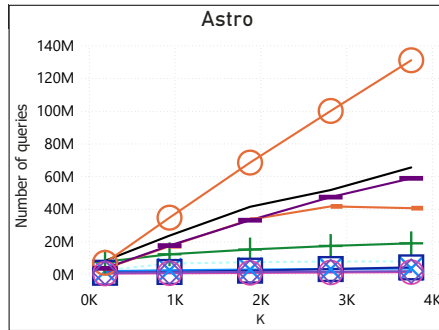
× AST + ATG □ BoostAdapt ○ FLS — FRG — IG ◇ LinAST ◇ LinATG - - - ParSSP — PITG



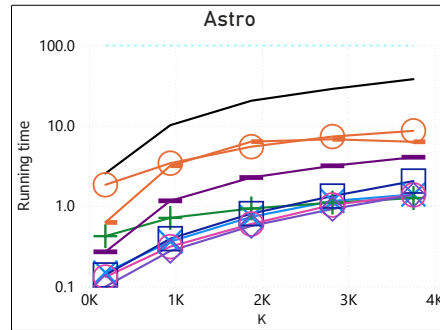
(a)



(b)



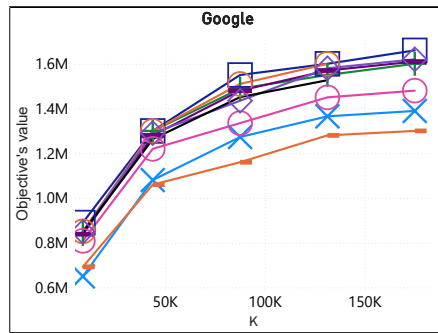
(c)



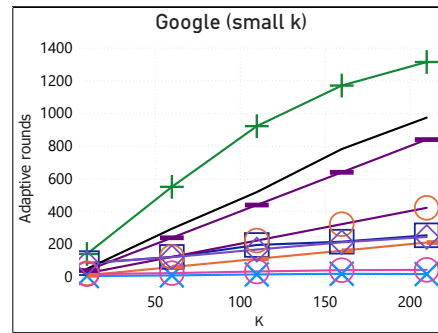
(d)

Figure 3: Performance of algorithms for Maximum Cut.

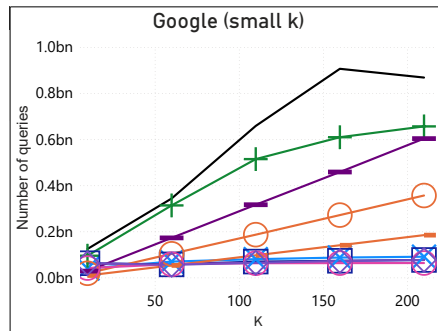
× AST + ATG □ BoostAdapt ○ FLS — FRG — IG ○ LinAST ◇ LinATG — PITG



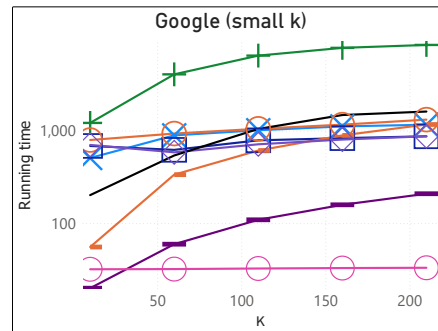
(a)



(b)



(c)



(d)

Figure 4: Performance of algorithms for Maximum Cut on large Google Dataset under small k settings.

× AST + ATG □ BoostAdapt ○ FLS — FRG — IG ○ LinAST ◇ LinATG — PITG

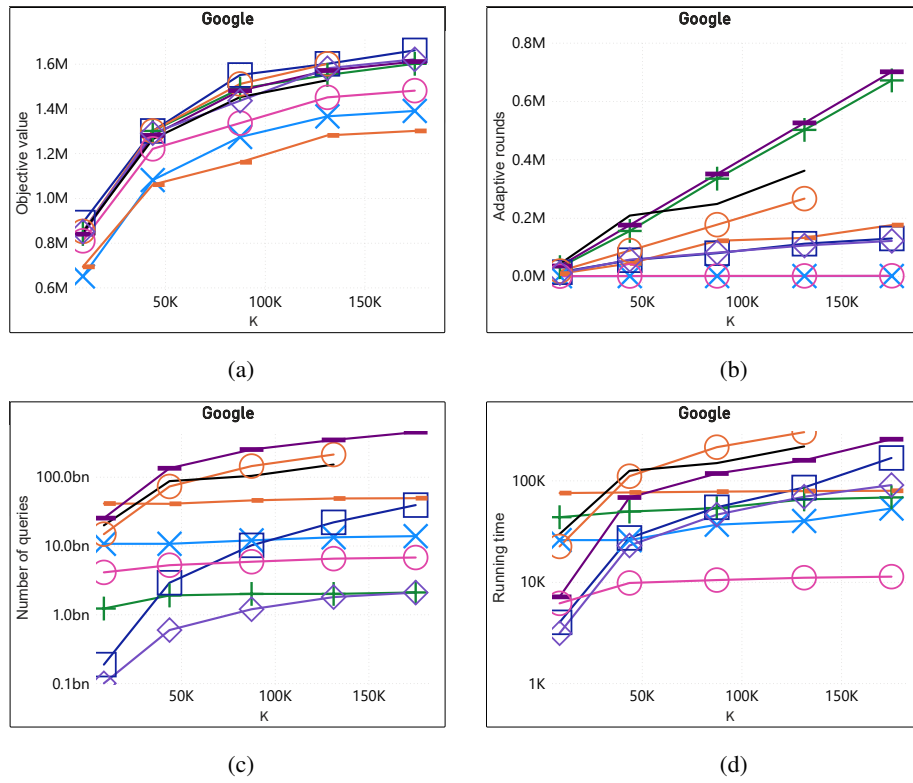


Figure 5: Performance of algorithms for Maximum Cut on large Google Dataset under large k settings.

local search method but this wasted an expensive query complexity of $O(n^4 \log n)$. Significantly, the elegant random greedy in [Buchbinder *et al.*, 2014] could provide the ratio of $1/e$ with $O(nk)$ query complexity. Instead of selecting an element with the best marginal gain as the naive greedy, it chose a uniformly random element from the set of k elements with the largest marginal gain. [Buchbinder *et al.*, 2015] then improved the query complexity to $O(n \log(1/\epsilon)/\epsilon^2)$ but still kept the same $1/e$ ratio. Recently, independent works [Chen *et al.*, 2025] and [Tukan *et al.*, 2024] improved the ratio to $0.385 - \epsilon$ in $O(n^2)$ queries and $O(n)$ adaptive complexity. To the best of our knowledge, the best ratio for SMC was 0.401 by [Buchbinder and Feldman, 2024]. However, this work used the multi-linear extensions method and used a considerably high query complexity of $O(n^5)$ [Chen and Kuhnle, 2023]. Finally, it must be emphasized that the above works weren't well parallelized due to their high adaptive complexities of $\Omega(n)$.